Porquinho: roteiro de testes manuais
Como usar este roteiro
Preparar o ambiente
Atenção
Epic 0 — Fundação do projeto e setup técnico
Story 0.1 — Inicializar o projeto frontend Vue
Story 0.2 — Inicializar o projeto backend Spring Boot
Story 0.3 — Configurar o PostgreSQL (Supabase) com Flyway
Story 0.4 — Configurar Tailwind CSS e shadcn-vue
Story 0.5 — Configurar o Redis para cache
Story 0.6 — Configurar CI/CD e containers Docker
Story 0.7 — Tokens de design e sistema de cores
Story 0.8 — Tipografia e espaçamento
Story 0.9 — Base de acessibilidade (WCAG 2.1 AA)
Epic 1 — Autenticação de usuário e gestão da conta
Story 1.1 — Cadastro com conta Google
Story 1.2 — Cadastro com e-mail e senha
Story 1.3 — Login com e-mail/senha ou Google
Story 1.4 — Recuperar a senha por e-mail
Story 1.5 — Ver sessões ativas e encerrar sessões remotas
Story 1.6 — Exportar todos os meus dados (LGPD)
Story 1.7 — Pedir a exclusão definitiva da conta (LGPD)
Epic 2 — Assinatura e cobrança (trial, Stripe)
Story 2.1 — Ativar automaticamente o trial gratuito de 30 dias
Story 2.2 — Assinar o plano mensal
Story 2.3 — Enviar lembretes de fim do trial
Story 2.4 — Cancelar a assinatura por conta própria
Story 2.5 — Reativar uma assinatura cancelada
Story 2.6 — Registrar os pagamentos mensais recorrentes
Story 2.7 — Tratar falhas de pagamento com novas tentativas e avisos
Epic 3 — Onboarding guiado
Story 3.1 — Boas-vindas e tutorial do orçamento por envelope
Story 3.2 — Adicionar contas bancárias no setup
Story 3.3 — Adicionar cartões de crédito no setup
Story 3.4 — Sugerir as categorias de gasto iniciais
Story 3.5 — Personalizar as categorias sugeridas
Story 3.6 — Primeira importação de transações guiada
Story 3.7 — Explicar a navegação em 3 camadas do Dashboard GPS
Story 3.8 — Criar a meta inicial de Reserva de Emergência
Epic 4 — Contas e cartões
Story 4.1 — Cadastrar várias contas bancárias
Story 4.2 — Cadastrar vários cartões de crédito
Story 4.3 — Editar informações de contas e cartões
Story 4.4 — Remover e restaurar contas e cartões
Story 4.5 — Registrar transferências entre contas próprias
Story 4.6 — Reconciliar o saldo da conta com o banco
Story 4.7 — Ver o histórico do saldo ao longo do tempo
Story 4.R — Correções da retro do Epic 4 (edição concorrente de conta)
Epic 5 — Importação de transações e classificação por IA
Story 5.1 — Upload de arquivos OFX dos 5 bancos

Story 5.2 — Upload de arquivos CSV dos 5 bancos
Story 5.3 — Detecção automática de formato e banco
Story 5.4 — Parsing de transações com tratamento robusto de erros
Story 5.5 — Detectar e descartar transações duplicadas
Story 5.6 — Identificar Pix e extrair detalhes
Story 5.7 — Detectar compras parceladas e criar parcelas futuras
Story 5.8 — Classificação automática com IA
Story 5.9 — Apresentar as transações classificadas para revisão
Story 5.10 — Revisar e ajustar classificações e concluir a importação
Story 5.11 — Aprender com as correções do usuário
Story 5.12 — Editar manualmente qualquer transação
Story 5.13 — Adicionar transações manualmente (despesas em dinheiro)

Epic 6 — Orçamento e categorias (método dos envelopes)

Story 6.1 — Criar categorias de gasto personalizadas
Story 6.2 — Organizar categorias em grupos

Apêndice: divergências entre specs e código

Prováveis bugs (confirmar)
Comportamento diferente do épico, por decisão ou por evolução
Não testável pela interface

Porquinho: roteiro de testes manuais

Stories concluídas até 9 de novembro de 2026: Epics 0 a 5 completos e Epic 6 até a Story 6.2.

Como usar este roteiro

- Siga as stories na ordem. Muitas dependem de dados criados antes: a conta da 1.x, o onboarding da 3.x, as contas e cartões da 4.x e as importações da 5.x.
- Em cada story, marque **Aprovado** ou **Reprovado** e anote o que viu. Se reprovar, informe a rota, o passo e a mensagem exibida.
- Os textos entre aspas são os rótulos e as mensagens que aparecem na tela.

Preparar o ambiente

Faça os passos abaixo **antes da Story 0.1**, nesta ordem, e deixe os três serviços rodando durante todos os testes.

1. Inicie o Redis: o container `porquinho-redis-dev` deve estar rodando na porta **6380**, e não na 6379, que pertence a outro projeto.
2. Inicie o backend com `cd porquinho-backend && JAVA_HOME=~/.sdkman/candidates/java/21.0.11-tem bash run-dev.sh`.
3. Inicie o frontend com `cd porquinho-frontend && npm run dev` e abra o endereço que o Vite mostrar.
4. Confira se tudo está no ar: `curl localhost:8080/actuator/health` deve responder `{"status":"UP"}`.
5. Para testar o Epic 2, deixe o encaminhamento de webhooks do Stripe rodando: `stripe listen --api-key <STRIPE_SECRET_KEY> --events checkout.session.completed,customer.subscription.deleted,invoice.paid,invoice.payment_failed --forward-to localhost:8080/api/v1/webhooks/stripe`.
6. Para testar a classificação por IA (5.8 a 5.11), defina `ANTHROPIC_API_KEY` no `.env` do backend.

Atenção

- O banco de desenvolvimento é o Postgres remoto (Supabase). Use e-mails de teste próprios, por exemplo `voce+teste1@gmail.com`, para não misturar com dados reais.
- A exclusão de conta (1.7) é definitiva depois do prazo de carência. Teste com uma conta descartável.
- Cartões de teste do Stripe: `4242 4242 4242 4242` aprova o pagamento. `4000 0000 0000 0341` aceita o cartão e recusa a cobrança.

Epic 0 — Fundação do projeto e setup técnico

> **Antes da Story 0.1, suba as três peças nesta ordem:** Redis (`docker start porquinho-redis-dev`), backend (`run-dev.sh`, e espere aparecer "Started PorquinhoBackendApplication") e frontend (`npm run dev`). As stories do Epic 0 conferem uma peça de cada vez, mas o app só funciona com as três no ar. Se só o frontend estiver rodando, o login mostra "Erro ao fazer login. Tente novamente." mesmo com a senha certa.

Story 0.1 — Inicializar o projeto frontend Vue

Objetivo: confirmar que o frontend Vue 3 + Vite sobe e tem as ferramentas de qualidade configuradas.

Pré-requisitos: Node 22+ instalado; `npm install` já rodado em `porquinho-frontend`; Redis e backend já no ar (veja o aviso no início do Epic 0).

Passos:

1. Com o frontend já no ar (aviso do início do Epic 0), olhe o terminal do `npm run dev`: ele mostra o endereço http://localhost:5173.
2. Abra http://localhost:5173 no navegador.
3. Em outro terminal, dentro de `porquinho-frontend`, rode `npm run lint` e `npm run test:unit -- --run`, sem parar o `npm run dev`.

Resultado esperado:

- O dev server sobe na porta 5173 e a raiz redireciona para `/login` (sem sessão) ou `/dashboard` (com sessão).
- Lint e testes unitários (Vitest) terminam sem erro.

Casos de erro a testar:

- Acessar uma rota protegida (ex.: `/dashboard`) sem estar logado → redireciona para `/login`.

□ Aprovado □ Reprovado — Observações: _____

Story 0.2 — Inicializar o projeto backend Spring Boot

Objetivo: confirmar que o backend sobe com JDK 21 e responde ao health check.

Pré-requisitos: `.env` do backend preenchido; Redis local rodando (ver Story 0.5).

Passos:

1. Com o backend já no ar (aviso do início do Epic 0), olhe o terminal do `run-dev.sh`.
2. Acesse http://localhost:8080/actuator/health.

Resultado esperado:

- O log mostra o Spring Boot iniciado com perfil `dev` na porta 8080.
- O health responde `{"status":"UP"}`.

Casos de erro a testar:

- Rodar com outro JDK (ex.: 17) → o build falha (alvo é Java 21).

□ Aprovado □ Reprovado — Observações: _____

Story 0.3 — Configurar o PostgreSQL (Supabase) com Flyway

Objetivo: confirmar que o backend conecta ao Postgres do Supabase e aplica as migrations.

Pré-requisitos: credenciais do Supabase no `.env`.

Passos:

1. No terminal do backend, procure no log de inicialização as linhas do Flyway.
2. Acesse http://localhost:8080/actuator/health.

Resultado esperado:

- O log mostra o Flyway validando ou aplicando migrations (“Successfully applied” quando há migration nova, ou “Schema … is up to date”).
- Health continua `{"status":"UP"}`.

Casos de erro a testar:

- Senha do banco errada no `.env` → o backend não sobe (erro de conexão do HikariCP no log).

□ Aprovado □ Reprovado — Observações: _____

Story 0.4 — Configurar Tailwind CSS e shadcn-vue

Objetivo: confirmar que os estilos Tailwind e os componentes shadcn-vue renderizam com o roxo de destaque.

Pré-requisitos: frontend rodando.

Passos:

1. Abra http://localhost:5173/login.
2. Abra http://localhost:5173/design-system.

Resultado esperado:

- Na tela de login, o card, os campos e o botão “Entrar” (roxo) aparecem estilizados.
- Os componentes em `src/components/ui/` (Button, Input, Card, Select, Dialog etc.) existem.

Casos de erro a testar:

- Nenhum caso de erro relevante pela UI.

□ Aprovado □ Reprovado — Observações: _____

Story 0.5 — Configurar o Redis para cache

Objetivo: confirmar que o backend usa o Redis local.

Pré-requisitos: Docker rodando.

Passos:

1. Rode `docker ps` e confira o container `porquinho-redis-dev` na porta 6380.
2. Confira que o `.env` do backend tem `REDIS_PORT=6380`.
3. Com o backend no ar, acesse http://localhost:8080/actuator/health.

Resultado esperado:

- Container ativo; health `{"status":"UP"}`.

Casos de erro a testar:

- Parar o container (`docker stop porquinho-redis-dev`) com o backend no ar → health passa a `{"status":"DOWN"}`; religar (`docker start porquinho-redis-dev`) → volta a `UP`.

□ Aprovado □ Reprovado — Observações: _____

Story 0.6 — Configurar CI/CD e containers Docker

Objetivo: confirmar que os Dockerfiles, o docker-compose e os workflows do GitHub Actions existem e funcionam.

Pré-requisitos: Docker e `gh` CLI instalados.

Passos:

1. Na raiz do repo, rode `docker compose config` (valida `docker-compose.yml` com os serviços frontend, backend, redis, nginx).
2. Rode `gh run list --limit 5` e confira as execuções de `frontend-ci.yml` e `backend-ci.yml`.

Resultado esperado:

- O `docker compose config` valida sem erro e não mostra segredos escritos direto no arquivo (vêm do `.env`).
- As últimas execuções de CI no `main` estão com sucesso.

Casos de erro a testar:

- Procurar senhas ou chaves escritas no `docker-compose.yml`, `nginx.conf` ou nos Dockerfiles → não deve haver nenhuma.

□ Aprovado □ Reprovado — Observações: _____

Story 0.7 — Tokens de design e sistema de cores

Objetivo: conferir a paleta de cores aprovada na página de design system.

Pré-requisitos: frontend rodando.

Passos:

1. Abra http://localhost:5173/design-system.
2. Confira as seções “Purple Scale (Primary)”, “Gray Scale”, “State Colors”, “Semantic Colors”, “Chart Colors” e “WCAG 2.1 AA Contrast Validation”.

Resultado esperado:

- O roxo primário é #9333EA (Purple 600); há 6 cores de gráfico; a tabela de contraste mostra os pares aprovados em AA.

Casos de erro a testar:

- Nenhum caso de erro relevante pela UI.

□ Aprovado □ Reprovado — Observações: _____

Story 0.8 — Tipografia e espaçamento

Objetivo: conferir a fonte Inter, a escala de texto e o espaçamento base de 4px.

Pré-requisitos: frontend rodando.

Passos:

1. Abra http://localhost:5173/design-system e role até “Sistema Tipografico” e “Espacamento e Layout”.
2. Nas DevTools, inspecione um texto e confira `font-family` começando por Inter.

Resultado esperado:

- “Escala Tipografica (Inter)” vai de text-xs a text-5xl; “Pesos Tipograficos” mostra 400, 500, 600 e 700.
- “Escala de Espacamento (base 4px)” e “Padroes de Uso Comuns” aparecem.

Casos de erro a testar:

- Nenhum caso de erro relevante pela UI.

□ Aprovado □ Reprovado — Observações: _____

Story 0.9 — Base de acessibilidade (WCAG 2.1 AA)

Objetivo: conferir foco visível, navegação por teclado e alvos de toque.

Pré-requisitos: frontend rodando.

Passos:

1. Abra http://localhost:5173/login e navegue só com Tab e Shift+Tab.
2. Abra http://localhost:5173/design-system e veja a seção “Accessibility Showcase”.
3. Rode o Lighthouse (DevTools > Lighthouse > Accessibility) em `/login`.

Resultado esperado:

- A ordem do Tab segue a tela: Email, Senha, olho de mostrar senha, “Esqueceu a senha?”, “Entrar”, “Entrar com Google”, “Cadastre-se”.
- Cada elemento focado mostra um contorno roxo visível.
- Lighthouse Accessibility acima de 90.

Casos de erro a testar:

- Pressionar Enter no botão do olho → alterna entre “Mostrar senha” e “Ocultar senha” sem enviar o formulário.

□ Aprovado □ Reprovado — Observações: _____

Epic 1 — Autenticação de usuário e gestão da conta

Story 1.1 — Cadastro com conta Google

Objetivo: criar uma conta usando o login do Google.

Pré-requisitos: frontend e backend rodando; uma conta Google que ainda não tem cadastro no Porquinho.

Passos:

1. Acesse `/register`.
2. Clique em “Cadastrar com Google”.
3. Escolha a conta Google e autorize.

Resultado esperado:

- Após o Google, abre `/auth/callback` com a mensagem “Finalizando seu cadastro…” e em seguida `/onboarding`.
- O usuário fica logado (recarregar a página não volta para `/login`).
- No banco: registro em `users` com `auth_provider` Google e `google_id` preenchido, e um evento `user_registration` em `audit_logs`.

Casos de erro a testar:

- Cancelar na tela do Google → volta ao app em `/auth/callback` com erro (ex.: “Nenhuma sessão encontrada. Por favor, tente novamente.”) e o botão “Voltar para registro”.
- Repetir o cadastro com a mesma conta Google → não cria um segundo usuário; entra normalmente.

□ Aprovado □ Reprovado — Observações: _____

Story 1.2 — Cadastro com e-mail e senha

Objetivo: criar uma conta com nome, e-mail e senha.

Pré-requisitos: um e-mail que ainda não tem cadastro e cuja caixa você consegue abrir.

Passos:

1. Acesse `/register`.
2. Preencha “Nome completo”, “Email”, “Senha” (ex.: `Teste1234`) e “Confirmar senha”.
3. Confira o indicador de força abaixo da senha.
4. Clique em “Cadastrar com Email”.
5. Abra o e-mail de verificação e clique no link.

Resultado esperado:

- O indicador mostra “Senha forte” e os quatro requisitos marcados com ✓ (“Mínimo de 8 caracteres”, “Pelo menos 1 letra maiúscula”, “Pelo menos 1 letra minúscula”, “Pelo menos 1 número”).
- Durante o envio, o botão mostra “Criando conta…”.
- O link do e-mail abre `/auth/verify-email` com “Sua conta foi ativada com sucesso” e o botão “Ir para Login”.

- No banco: registro em `users` e evento `user_registration` em `audit_logs`.

Casos de erro a testar:

- Senha `abc12345` (sem maiúscula) → requisito fica com ○, indicador mostra “Senha fraca” ou “Senha média” e “Cadastrar com Email” fica desabilitado.
- “Confirmar senha” diferente → aparece “As senhas não coincidem” e o botão fica desabilitado.
- E-mail já cadastrado → deveria aparecer “Este email já está cadastrado.” (ver observação: o Supabase pode esconder isso).
- Abrir de novo um link de verificação já usado ou vencido → “Não foi possível verificar seu email” com o botão “Voltar para Registro”.

□ Aprovado □ Reprovado — Observações: _____

Story 1.3 — Login com e-mail/senha ou Google

Objetivo: entrar na conta, com proteção contra tentativas repetidas.

Pré-requisitos: conta com e-mail verificado (Story 1.2) e uma conta Google cadastrada (Story 1.1).

Passos:

1. Acesse `/login`.
2. Preencha “Email” e “Senha” e clique em “Entrar”.
3. Faça logout, volte a `/login` e clique em “Entrar com Google”.

Resultado esperado:

- Durante o envio, o botão mostra “Entrando…”.
- Login com sucesso leva a `/onboarding` (ou ao `/dashboard`, se o onboarding já foi concluído).
- No banco: evento `user_login` em `audit_logs` com IP, e uma linha nova em `user_sessions`.

Casos de erro a testar:

- Senha errada → após cerca de 1 segundo aparece “Email ou senha inválidos.”.
- 5 senhas erradas seguidas e uma 6ª tentativa → “Conta temporariamente bloqueada devido a múltiplas tentativas falhas. Tente novamente em 15 minutos.” (nem a senha correta entra durante o bloqueio).
- Mais de 5 tentativas em 1 minuto com e-mails variados → “Muitas tentativas de login. Aguarde 1 minuto.”.
- Conta com e-mail não verificado → “Email não verificado. Verifique sua caixa de entrada e clique no link de confirmação.”.

□ Aprovado □ Reprovado — Observações: _____

Story 1.4 — Recuperar a senha por e-mail

Objetivo: redefinir a senha pelo link enviado por e-mail.

Pré-requisitos: conta com e-mail e senha cuja caixa você consegue abrir.

Passos:

1. Em `/login`, clique em “Esqueceu a senha?”.
2. Digite o e-mail e clique em “Enviar link de redefinição”.

3. Abra o e-mail e clique no link (abre `/auth/reset-password`).
4. Preencha “Nova senha” e “Confirmar senha” e clique em “Redefinir senha”.
5. Entre com a nova senha em `/login`.

Resultado esperado:

- Após o envio: “Verifique seu email” / “Se esse email existir, você receberá instruções para redefinir sua senha em até 5 minutos.”.
- Os requisitos “Pelo menos 8 caracteres”, “Uma letra maiúscula”, “Uma letra minúscula” e “Um número” ficam com ✓.
- Após redefinir: “Senha redefinida com sucesso!” / “Redirecionando para login…” e, em 3 segundos, vai para `/login`.
- O login com a senha antiga falha e com a nova funciona.
- No banco: eventos `password_reset_requested` e `password_reset_completed` em `audit_logs`.

Casos de erro a testar:

- E-mail não cadastrado → mesma mensagem de sucesso genérica (não revela se o e-mail existe).
- Abrir de novo o link já usado → “Link de redefinição expirado ou inválido. Solicite um novo.” com o link “Solicitar novo link”.
- Nova senha igual à atual → “A nova senha deve ser diferente da senha anterior.”.
- Senhas diferentes nos dois campos → “As senhas não coincidem”.

□ Aprovado □ Reprovado — Observações: _____

Story 1.5 — Ver sessões ativas e encerrar sessões remotas

Objetivo: listar os dispositivos conectados e encerrar sessões de outros dispositivos.

Pré-requisitos: a mesma conta logada em dois navegadores diferentes (ex.: Chrome e uma janela anônima ou Safari).

Passos:

1. No navegador A, vá ao `/dashboard` e clique em “Sessoes ativas” (abre `/auth/sessions`).
2. Confira a lista.
3. Num card que não é o atual, clique em “Encerrar sessao” e depois em “Confirmar”.
4. No navegador B, faça qualquer ação que chame o backend (ex.: recarregar `/dashboard`).
5. Entre de novo no navegador B; no A, clique em “Encerrar todas as outras sessoes” e em “Confirmar”.

Resultado esperado:

- Cada card mostra o navegador (ex.: “Google Chrome”), “IP:”, “Ultimo acesso:” e “Criada em:”; a sessão atual tem o selo “Dispositivo atual” e não tem o botão “Encerrar sessao”.
- O diálogo “Encerrar sessao” pergunta “Tem certeza que deseja encerrar esta sessao? O dispositivo sera desconectado imediatamente.”.
- Após confirmar, a sessão some da lista e o navegador B perde o acesso (as chamadas ao backend passam a falhar com 401).
- “Encerrar todas as outras sessoes” deixa só a sessão atual e o botão some.
- No banco: eventos `remote_logout` em `audit_logs`.

Casos de erro a testar:

- Clicar em “Cancelar” no diálogo → nada é encerrado.
- Backend parado e recarregar a página → “Nao foi possivel carregar as sessoes. Tente novamente.” com o botão “Tentar novamente”.
- Sessão com mais de 7 dias sem uso não aparece na lista (não dá para testar pela UI sem mexer no banco).

□ Aprovado □ Reprovado — Observações: _____

Story 1.6 — Exportar todos os meus dados (LGPD)

Objetivo: gerar e baixar um arquivo JSON com os dados do usuário.

Pré-requisitos: usuário logado.

Passos:

1. No `/dashboard`, clique em “Exportar meus dados” (abre `/auth/data-export`).
2. Clique em “Exportar Meus Dados” e, no diálogo “Confirmar exportacao de dados”, clique em “Confirmar”.
3. Aguarde o status mudar (a tela se atualiza sozinha).
4. Clique em “Baixar arquivo” e abra o JSON.
5. Confira a caixa de e-mail.

Resultado esperado:

- Durante o processamento aparece “Exportacao em andamento” / “Seus dados estao sendo coletados e preparados para download…”.
- Depois aparece “Exportacao pronta!” com “Expira em:” e “O link expira apos 24 horas ou apos o primeiro download.”.
- O JSON tem metadados (data da exportação, id do usuário, versão, tabelas exportadas) e os dados de perfil, sessões e logs de auditoria (e as tabelas dos épicos seguintes, se já incluídas). Não contém `jwt_token_hash`, `failed_login_attempts`, `locked_until`, `deleted_at` nem `deleted_by`.
- Chega um e-mail (Resend) com o link de download e o aviso de 24h.
- No banco: evento `data_export_requested` em `audit_logs`.

Casos de erro a testar:

- Abrir o mesmo link de download pela segunda vez → o arquivo não é mais entregue (erro 404).
- Pedir uma segunda exportação dentro de 1 hora → “Limite de exportacao atingido. Tente novamente em 1 hora.”.
- Depois do download, voltar à página → aparece de novo o card “Solicitar exportacao”.

□ Aprovado □ Reprovado — Observações: _____

Story 1.7 — Pedir a exclusão definitiva da conta (LGPD)

Objetivo: pedir a exclusão da conta com confirmação por frase e logout imediato.

Pré-requisitos: uma conta de teste descartável, logada. Atenção: a conta fica bloqueada e é apagada em 30 dias.

Passos:

1. No `/dashboard`, em “Zona de perigo”, clique em “Excluir minha conta” (abre `/auth/account-deletion`).

2. Leia o card “Excluir minha conta permanentemente”.
3. Clique em “Excluir minha conta”.
4. No diálogo “Confirmar exclusao de conta”, digite `EXCLUIR MINHA CONTA`.
5. Clique em “Confirmar exclusao”.

Resultado esperado:

- O card lista o que será removido (perfil, sessões, histórico de auditoria, exportações) e avisa sobre a retenção de “30 dias” e que depois “nao havera como recuperar a conta”.
- “Confirmar exclusao” fica desabilitado até a frase estar correta.
- Após confirmar: logout e redirecionamento para `/login` com “Sua conta foi marcada para exclusao. Os dados serao removidos definitivamente em 30 dias.”.
- Chega um e-mail de confirmação com a data do pedido e o prazo de 30 dias.
- No banco: `users.deleted_at` preenchido, `account_deletions` com `PENDING_PURGE` e evento `account_deletion_requested` em `audit_logs`.

Casos de erro a testar:

- Digitar `excluir minha conta` (minúsculas) → “Confirmar exclusao” continua desabilitado.
- Fechar com “Cancelar” → nada acontece e a conta continua ativa.
- Tentar entrar de novo com a conta excluída → o acesso deve ser negado (ver observação: pelo código, o login pode parecer funcionar).
- Tentar se cadastrar de novo com o mesmo e-mail → o backend recusa com “Esta conta está em exclusão. Aguarde a conclusão para se cadastrar de novo.” (o frontend pode não mostrar esse texto).

□ Aprovado □ Reprovado — Observações: _____

Epic 2 — Assinatura e cobrança (trial, Stripe)

Story 2.1 — Ativar automaticamente o trial gratuito de 30 dias

Objetivo: confirmar que todo cadastro novo ganha 30 dias de trial com acesso completo, e que o acesso é bloqueado quando o trial vence.

Pré-requisitos: backend (`JAVA_HOME=~/.sdkman/candidates/java/21.0.11-tem bash run-dev.sh`) e frontend rodando; um e-mail que ainda não tem conta; acesso ao SQL Editor do Supabase.

Passos:

1. Em `/register` , cadastre um usuário novo.
2. Confira que o app leva para `/onboarding` .
3. Abra `/dashboard` .
4. No Supabase, rode: `SELECT plan_type, status, trial_start_date, trial_end_date FROM subscriptions WHERE user_id = (SELECT id FROM users WHERE email = '<seu-email>');`
5. Rode: `SELECT event_type, metadata FROM audit_logs WHERE user_id = (SELECT id FROM users WHERE email = '<seu-email>') AND event_type = 'trial_started';`
6. Simule o fim próximo do trial: `UPDATE subscriptions SET trial_end_date = now() + interval '12 hours' WHERE user_id = (SELECT id FROM users WHERE email = '<seu-email>');` e recarregue `/dashboard` .
7. Simule o trial vencido: `UPDATE subscriptions SET trial_end_date = now() - interval '1 minute' WHERE user_id = (...mesmo filtro...);` e recarregue `/dashboard` .
8. Com o trial vencido, clique em “Contas bancárias”.

Resultado esperado:

- Passo 3: ao lado do título “Dashboard”, o badge verde “Trial: 30 dias restantes” e o link “Assinar”.
- Passo 4: uma linha `TRIAL` / `ACTIVE` , com `trial_end_date` = `trial_start_date` + 30 dias.
- Passo 5: um registro `trial_started` com o `trial_end_date` no metadata.
- Passo 6: badge “Trial: 1 dia restante” (singular).
- Passo 7: badge vermelho “Trial expirado” e o link “Assinar”.
- Passo 8: o app redireciona para `/assinatura` (a API responde 402).

Casos de erro a testar:

- Fazer login de novo com o mesmo usuário (ou repetir o cadastro com Google) → o trial não é recriado nem reiniciado (`trial_end_date` não muda).
- Trial vencido + abrir `/auth/data-export` → a página funciona (rotas LGPD ficam liberadas mesmo sem assinatura).
- Trial vencido + abrir `/assinatura` → a página carrega e mostra o botão “Assinar” (as rotas de assinatura nunca são bloqueadas).

□ Aprovado □ Reprovado — Observações: _____

Story 2.2 — Assinar o plano mensal

Objetivo: assinar o plano mensal (R$ 19,90/mês) pelo Stripe Checkout, durante o trial e depois dele.

Pré-requisitos: `stripe listen --api-key <STRIPE_SECRET_KEY> --events checkout.session.completed,customer.subscription.deleted,invoice.paid,invoice.payment_failed --forward-to localhost:8080/api/v1/webhooks/stripe` rodando, com o `whsec_...` em `STRIPE_WEBHOOK_SECRET`; usuário em trial ativo (Story 2.1); SMTP do Resend configurado no `.env`.

Passos:

1. No `/dashboard`, clique no link "Assinar".
2. Em `/assinatura`, confira o card "Plano mensal" com "R$ 19,90/mês" e o texto "Assinando agora, você mantém os dias grátis do trial: a primeira cobrança só acontece quando ele terminar."
3. Clique em "Assinar". O botão muda para "Redirecionando…" e abre o Checkout do Stripe.
4. Pague com `4242 4242 4242 4242`, qualquer validade futura e qualquer CVC.
5. Aguarde a volta para `/assinatura?status=sucesso`.
6. Confira no terminal do `stripe listen` o `checkout.session.completed` com resposta 200.
7. No Supabase: `SELECT plan_type, status, next_billing_date, trial_end_date, stripe_customer_id, stripe_subscription_id, payment_method_id FROM subscriptions WHERE user_id = (...)`.
8. Volte ao `/dashboard`.

Resultado esperado:

- Passo 5: faixa verde "Assinatura confirmada! A primeira cobrança acontece no fim do trial." Em até ~10 s (a tela consulta a cada 2 s) aparecem o badge "Assinatura ativa" e "Sua assinatura está ativa. Obrigado por apoiar o Porquinho!".
- Passo 7: `MONTHLY` / `ACTIVE`, ids do Stripe preenchidos e `next_billing_date` igual ao antigo `trial_end_date`.
- No Stripe Dashboard (modo teste): uma assinatura `trialing` de R$ 19,90.
- E-mail "Porquinho - Assinatura ativada", com "Próxima cobrança: dd/mm/aaaa".
- Audit `subscription_activated` gravado.
- Passo 8: badge "Assinatura ativa", sem o link "Assinar".

Casos de erro a testar:

- No Checkout, clicar em voltar/cancelar → `/assinatura?status=cancelado` com "Assinatura não concluída. Nada foi cobrado e seu trial continua valendo."; nada muda no banco.
- Já com a assinatura ativa, abrir `/assinatura` → não aparece o botão "Assinar", só "Cancelar assinatura". Via API, `POST /api/v1/subscriptions/checkout` responde 409 "Você já possui uma assinatura ativa."
- Deixar `STRIPE_SECRET_KEY` vazio no `.env`, reiniciar o backend e clicar em "Assinar" → "Pagamento indisponível no momento. Tente novamente em alguns minutos."; o trial continua intacto.
- `curl -X POST localhost:8080/api/v1/webhooks/stripe -d '{}'` (sem assinatura Stripe) → 400 e nada muda no banco.

□ Aprovado □ Reprovado — Observações: _____

Story 2.3 — Enviar lembretes de fim do trial

Objetivo: enviar os e-mails "Seu trial termina em 7 dias" e "Seu trial termina amanhã", cada um uma única vez, só para quem está em trial.

Pré-requisitos: usuário em `TRIAL` / `ACTIVE` (sem assinatura); SMTP do Resend configurado. O job roda todo dia às 9h (America/Sao_Paulo) **e também quando o backend sobe**, então para testar basta ajustar a data no banco e reiniciar o backend.

Passos:

1. No Supabase: `UPDATE subscriptions SET trial_end_date = now() + interval '5 days', trial_reminder_7d_sent_at = NULL, trial_reminder_1d_sent_at = NULL WHERE user_id = (SELECT id FROM users WHERE email = '<seu-email>');`
2. Reinicie o backend e procure no log a linha “Trial reminder job done”.
3. Confira a caixa de entrada.
4. Reinicie o backend de novo.
5. Rode: `UPDATE subscriptions SET trial_end_date = now() + interval '20 hours' WHERE user_id = (...);` e reinicie o backend.
6. Confira a caixa de entrada e rode `SELECT trial_reminder_7d_sent_at, trial_reminder_1d_sent_at FROM subscriptions WHERE user_id = (...);`
7. Rode `SELECT event_type, metadata FROM audit_logs WHERE user_id = (...) AND event_type = 'trial_reminder_sent';`

Resultado esperado:

- Passo 3: e-mail “Porquinho - Seu trial termina em 5 dias” (ou “em N dias”, conforme a data em São Paulo), com a data de fim, o preço R$ 19,90/mês e o link “Assinar agora: http://localhost:5173/assinatura”.
- Passo 4: nenhum e-mail novo.
- Passo 6: e-mail “Porquinho - Seu trial termina amanhã” (ou “hoje”, se o fim cair no mesmo dia em São Paulo). As duas colunas `_sent_at` ficam preenchidas.
- Passo 7: dois registros, `{"reminder":"7d"}` e `{"reminder":"1d"}`.
- O link do e-mail abre `/assinatura`.

Casos de erro a testar:

- Usuário que já assinou (`MONTHLY`) com `trial_end_date` em 5 dias → reiniciar o backend não envia nada.
- Zerar as colunas `_sent_at` e colocar o fim do trial direto em 20 horas → sai só o lembrete de 1 dia; o de 7 dias nunca é enviado.
- Trial já vencido → nenhum lembrete.
- SMTP não configurado → log “SMTP not configured - skipping trial reminder email”; a coluna fica marcada e o lembrete não é reenviado depois.

□ Aprovado □ Reprovado — Observações: _____

Story 2.4 — Cancelar a assinatura por conta própria

Objetivo: cancelar o plano mensal sem nova cobrança, mantendo o acesso até o fim do período pago.

Pré-requisitos: usuário com `MONTHLY` / `ACTIVE` (Story 2.2); `stripe listen` rodando.

Passos:

1. Abra `/assinatura` e clique em “Cancelar assinatura”.
2. Leia o modal “Cancelar assinatura?” e clique em “Manter assinatura”.
3. Clique de novo em “Cancelar assinatura” e, no modal, confirme em “Cancelar assinatura” (o botão mostra “Cancelando…”).
4. Abra `/dashboard`.
5. No Supabase, confira `status`, `cancellation_date` e `next_billing_date` em `subscriptions`.
6. No Stripe Dashboard, abra a assinatura.

7. Ainda no Stripe, cancele a assinatura imediatamente (ou rode `stripe subscriptions cancel <sub_id> --api-key <STRIPE_SECRET_KEY>`) e recarregue `/dashboard`.

Resultado esperado:

- Passo 2: o texto do modal é “Tem certeza? Você perde o acesso ao fim do período atual (dd/mm/aaaa). Nenhuma nova cobrança será feita.”. “Manter assinatura” fecha o modal sem mudar nada.
- Passo 3: “Assinatura cancelada — ativa até dd/mm/aaaa. Nenhuma nova cobrança será feita.” e o botão “Reativar assinatura”.
- Passo 4: badge amarelo “Assinatura cancelada (ativa até dd/mm/aaaa)”; o app continua acessível.
- Passo 5: `CANCELLED`, `cancellation_date` preenchida e `next_billing_date` inalterada. Audit `subscription_cancelled`.
- Passo 6: a assinatura aparece como “cancela em” na data do fim do período.
- E-mail “Porquinho - Assinatura cancelada”, com “Você continua com acesso completo até dd/mm/aaaa”.
- Passo 7: webhook `customer.subscription.deleted` → linha `EXPIRED`; badge “Assinatura expirada” e link “Reativar”; abrir “Contas bancárias” leva para `/assinatura`.

Casos de erro a testar:

- Cancelar em duas abas: confirmar na 1ª e depois na 2ª → a 2ª só recarrega e mostra o estado cancelado, sem erro (a API responde 409 “Sua assinatura já está cancelada.”).
- Usuário em trial sem plano mensal → `/assinatura` não mostra “Cancelar assinatura” (a API responderia 409 “Não há assinatura mensal ativa para cancelar.”).
- Stripe indisponível (`STRIPE_SECRET_KEY` inválida + reinício) ao confirmar → “Não foi possível cancelar agora. Nada foi alterado; tente novamente em alguns minutos.”; a linha continua `ACTIVE`.
- Cancelada com acesso + `POST /api/v1/subscriptions/checkout` → 409 “Sua assinatura cancelada ainda está ativa até o fim do período.”

□ Aprovado □ Reprovado — Observações: _____

Story 2.5 — Reativar uma assinatura cancelada

Objetivo: reativar uma assinatura cancelada, sem cobrança se ainda estiver no período pago, ou pelo Checkout (com cobrança imediata) se já tiver expirado.

Pré-requisitos: cenário A: assinatura `CANCELLED` com acesso (Story 2.4, passo 3). Cenário B: assinatura `EXPIRED` (Story 2.4, passo 7). `stripe listen` rodando.

Passos:

1. Cenário A: em `/assinatura`, leia “Reativando agora, nada é cobrado hoje: a próxima cobrança acontece em dd/mm/aaaa.” e clique em “Reativar assinatura” (o botão mostra “Reativando…”).
2. Confira o banco e o Stripe Dashboard.
3. Cenário B: no `/dashboard`, clique no link “Reativar”.
4. Em `/assinatura` (badge “Assinatura expirada”), clique no botão “Reativar assinatura”. O Checkout abre.
5. Pague com `4242 4242 4242 4242`.
6. Na volta para `/assinatura?status=sucesso`, aguarde o badge.

Resultado esperado:

- Passo 1: badge “Assinatura ativa” e “Sua assinatura está ativa. Obrigado por apoiar o Porquinho!”.
- Passo 2: linha `MONTHLY` / `ACTIVE`, sem novo `billing_transactions`. No Stripe, a assinatura deixa de mostrar “cancela em”. Audit `subscription_reactivated` e e-mail “Porquinho - Assinatura reativada”.

- Passo 6: “Assinatura confirmada!” e depois “Assinatura ativa”. O Stripe cobra R$ 19,90 na hora (sem trial). Chegam os e-mails “Porquinho - Assinatura reativada” e “Porquinho - Pagamento confirmado”, e é gravada uma linha em `billing_transactions`.

Casos de erro a testar:

- Cenário A em duas abas: reativar na 1ª e depois na 2ª → a 2ª mostra o estado ativo, sem erro.
- Cenário A depois de cancelar a assinatura direto no Stripe, sem o webhook ter chegado (pare o `stripe listen` antes) → “Não foi possível reativar esta assinatura. Assine novamente para voltar a ter acesso.”
- Stripe indisponível no cenário A → “Não foi possível reativar agora. Nada foi alterado; tente novamente em alguns minutos.”
- Cenário B com o cartão `4000 0000 0000 0341` → o próprio Checkout recusa; a linha continua `EXPIRED`.

□ Aprovado □ Reprovado — Observações: _____

Story 2.6 — Registrar os pagamentos mensais recorrentes

Objetivo: a cada fatura paga pelo Stripe, gravar o pagamento, atualizar a próxima cobrança e enviar o recibo. A cobrança em si é feita pelo Stripe; o porquinho não tem job próprio.

Pré-requisitos: usuário que assinou durante o trial (Story 2.2, assinatura `trialing` no Stripe); `stripe listen` rodando; o `stripe_subscription_id` (`sub_...`) da tabela `subscriptions`.

Passos:

1. Antecipe a primeira cobrança encerrando o trial no Stripe: `stripe subscriptions update <sub_id> -d trial_end=now --api-key <STRIPE_SECRET_KEY>`.
2. Confira no `stripe listen` o `invoice.paid` com resposta 200.
3. No Supabase: `SELECT amount, currency, status, stripe_transaction_id, paid_at FROM billing_transactions WHERE user_id = (...);` e `SELECT next_billing_date FROM subscriptions WHERE user_id = (...);`
4. Confira a caixa de entrada.
5. Reenvie o mesmo evento: `stripe events resend <evt_id do invoice.paid> --api-key <STRIPE_SECRET_KEY>`.
6. Repita a consulta do passo 3 e confira a caixa de entrada.

Resultado esperado:

- Passo 3: uma linha com `19.90`, `brl`, `PAID` e o id da fatura (`in_...`); `next_billing_date` avança para cerca de 1 mês depois de hoje.
- Passo 4: e-mail “Porquinho - Pagamento confirmado”, com “Valor: R$ 19,90”, a data do pagamento e a próxima cobrança.
- Audit `payment_success`.
- Passo 6: nenhuma linha, e-mail ou audit duplicado.

Casos de erro a testar:

- Assinar durante o trial (fatura de R$ 0) → nenhuma linha em `billing_transactions` e nenhum recibo.
- Assinar com o trial vencido ou reativar uma expirada (Story 2.5 B) → a 1ª fatura cobrada na hora também é gravada e gera recibo.
- Renovação real de mês a mês: só dá para testar com um Test Clock do Stripe (cliente criado no clock). Opcional; o passo 1 já cobre o mesmo webhook.

□ Aprovado □ Reprovado — Observações: _____

Story 2.7 — Tratar falhas de pagamento com novas tentativas e avisos

Objetivo: quando a cobrança falha, avisar o usuário (banner e e-mail), manter o acesso enquanto o Stripe tenta de novo, reativar ao pagar e suspender quando todas as tentativas falham.

Pré-requisitos: usuário novo em trial; `stripe listen` rodando. No Stripe Dashboard (Billing → Revenue recovery → Retries): 2 novas tentativas, 3 e 10 dias depois, e cancelar a assinatura após a última.

Passos:

1. Em `/assinatura`, clique em "Assinar" e, no Checkout, use o cartão `4000 0000 0000 0341` (é aceito no cadastro e recusado na cobrança).
2. Force a cobrança: `stripe subscriptions update <sub_id> -d trial_end=now --api-key <STRIPE_SECRET_KEY>`.
3. Confira o `invoice.payment_failed` no `stripe listen` e abra `/dashboard`.
4. Abra `/assinatura`.
5. No Supabase: `SELECT status, failed_payment_attempts, payment_update_url FROM subscriptions WHERE user_id = (...);`
6. Clique em "Atualizar pagamento" e pague a fatura no Stripe com `4242 4242 4242 4242`.
7. Volte ao `/dashboard`.
8. Para testar a suspensão, repita os passos 1–3 com outro usuário e depois rode `stripe subscriptions cancel <sub_id> --api-key <STRIPE_SECRET_KEY>`, que simula o Stripe desistindo após a última tentativa.

Resultado esperado:

- Passo 3: badge "Pagamento falhou" e banner com o título "Pagamento falhou" e o texto "Não conseguimos cobrar sua mensalidade. Vamos tentar de novo nos próximos dias; atualize o pagamento para não perder o acesso." com o botão "Atualizar pagamento". O app continua acessível.
- Passo 4: "Não conseguimos cobrar sua última mensalidade. Vamos tentar de novo automaticamente e seu acesso continua enquanto isso." com os botões "Atualizar pagamento" e "Cancelar assinatura".
- Passo 5: `PAYMENT_FAILED`, `failed_payment_attempts = 1` e o link `https://invoice.stripe.com/...`. Audit `payment_failed`.
- E-mail "Porquinho - Pagamento falhou", com o link da fatura.
- Passos 6–7: `invoice.paid` → `ACTIVE`, contador zerado e link limpo; audit `payment_recovered`; e-mail "Porquinho - Pagamento confirmado"; o banner some e aparece "Assinatura ativa".
- Passo 8: `SUSPENDED`, badge "Assinatura suspensa" e link "Reativar"; telas do app levam para `/assinatura`; e-mail "Porquinho - Assinatura suspensa"; audit `subscription_suspended`; a fatura em aberto fica anulada (void) no Stripe.

Casos de erro a testar:

- Em `PAYMENT_FAILED`, clicar em "Cancelar assinatura" → o modal diz "Tem certeza? Como o último pagamento falhou, o acesso termina agora. Não haverá novas tentativas de cobrança."; ao confirmar: "Assinatura cancelada. Seu acesso foi encerrado e nenhuma nova cobrança será feita.", linha `EXPIRED` e e-mail "Porquinho - Assinatura cancelada" com "Seu acesso ao app foi encerrado hoje." (sem e-mail de suspensão).
- `stripe events resend` do mesmo `invoice.payment_failed` → nada muda (contador, audit e e-mail não se repetem).

- Em `PAYMENT_FAILED`, `POST /api/v1/subscriptions/checkout` → 409 “Seu último pagamento falhou. Pague a fatura pendente para manter sua assinatura.”
- Usuário `SUSPENDED` clica em “Reativar assinatura” em `/assinatura` → abre um Checkout novo (fluxo 2.5 B).

□ Aprovado □ Reprovado — Observações: _____

Epic 3 — Onboarding guiado

Story 3.1 — Boas-vindas e tutorial do orçamento por envelope

Objetivo: mostrar a boas-vindas e o tutorial de 4 telas (passo 1 de 8), com opção de pular, e salvar o progresso no servidor.

Pré-requisitos: usuário recém-cadastrado (o cadastro e o login levam para `/onboarding`).

Passos:

1. Em `/onboarding`, confira "Passo 1 de 8" e o título "Bem-vindo ao porquinho!".
2. Clique em "Próximo" e passe pelas telas "O que é o orçamento por envelope?", "Como funciona no porquinho", "Por que funciona?" e "Pronto para começar?".
3. Em alguma tela, clique em "Voltar" e confira que volta uma tela.
4. Na última tela, clique em "Próximo".
5. Com outro usuário novo, clique em "Pular tutorial", depois em "Ver o tutorial", e de novo em "Pular tutorial" → "Pular".
6. Recarregue a página (F5) no passo 2.

Resultado esperado:

- Passo 2: cada tela tem diagrama (salário de R$ 3.000 em 3 envelopes; "Envelope Mercado" com "Ainda sobram R$ 680"; 3 motivos) e as bolinhas de progresso.
- Passo 4: vai para "Passo 2 de 8" — "Adicione suas contas bancárias".
- Passo 5: o modal "Pular o tutorial?" diz "Você pode seguir direto para a configuração da sua conta. O tutorial leva só 1 minuto."; "Ver o tutorial" fecha o modal e "Pular" leva ao passo 2.
- Passo 6: a página reabre no passo 2 (progresso salvo em `onboarding_progress`).

Casos de erro a testar:

- Backend parado ao clicar em "Próximo" na última tela → toast "Não foi possível salvar seu progresso. Verifique sua conexão e tente novamente." e a tela não avança.
- Backend parado ao abrir `/onboarding` → toast "Não foi possível carregar seu progresso. Você pode continuar o tutorial normalmente."
- Duplo clique rápido em "Pular" → um único avanço (o botão mostra "Salvando…").

□ Aprovado □ Reprovado — Observações: _____

Story 3.2 — Adicionar contas bancárias no setup

Objetivo: cadastrar uma ou mais contas bancárias no passo 2, ou pular e ganhar a conta "Minha Conta Principal".

Pré-requisitos: usuário no passo 2 (Story 3.1 concluída).

Passos:

1. Confira o texto "Nenhuma conta adicionada ainda." e o botão "Próximo" desabilitado, com o aviso "Adicione ao menos uma conta para continuar. Se pular, criamos a"Minha Conta Principal" com saldo zero para você."

2. Preencha “Nome da conta” = `Nubank`, “Banco (opcional)” = escolha da lista, “Tipo da conta” = “Conta corrente” e “Saldo atual da conta (R$)” = `1.234,56`. Clique em “Adicionar conta”.
3. Adicione uma segunda conta do tipo “Poupança” com saldo `-150,00`.
4. Remova a segunda conta pelo botão “Remover”.
5. Clique em “Próximo”.
6. Abra `/contas` em outra aba e confira o saldo.
7. Com outro usuário no passo 2, sem nenhuma conta, clique em “Pular por agora”.

Resultado esperado:

- Passo 2: a conta aparece em “Suas contas (1)”: “Nubank” e, abaixo, “Conta corrente · ”.
- Passo 4: a lista volta para 1 conta.
- Passo 5: vai para “Passo 3 de 8”.
- Passo 6: o saldo atual é igual ao saldo inicial informado (R$ 1.234,56).
- Passo 7: vai para o passo 3 e em `/contas` existe “Minha Conta Principal”, conta corrente com saldo R$ 0,00.

Casos de erro a testar:

- Nome vazio → “Informe o nome da conta”.
- Saldo vazio → “Informe o saldo inicial (use 0,00 se não souber)”; saldo `12,345` → “Digite o saldo no formato 1.234,56 (até 2 casas depois da vírgula)”.
- No passo 3, clicar em “Voltar” e depois em “Próximo” no passo 2 → volta ao passo 3 sem criar nada de novo.
- “Pular por agora” já tendo contas cadastradas → avança sem criar “Minha Conta Principal”.

□ Aprovado □ Reprovado — Observações: _____

Story 3.3 — Adicionar cartões de crédito no setup

Objetivo: cadastrar cartões no passo 3, que é opcional.

Pré-requisitos: usuário no passo 3.

Passos:

1. Confira o título “Adicione seus cartões de crédito” e “Nenhum cartão adicionado ainda.”
2. Preencha “Nome do cartão” = `Nubank Roxinho`, banco opcional, “Limite do cartão (R$)” = `5.000,00` e “Dia do vencimento” = `10`. Clique em “Adicionar cartão”.
3. Remova o cartão pelo botão “Remover” e adicione de novo.
4. Clique em “Próximo”.
5. Abra `/cartoes` e confira o cartão.
6. Com outro usuário no passo 3, clique em “Próximo” sem cartões (ou em “Pular por agora”).

Resultado esperado:

- Passo 2: “Seus cartões (1)” mostra “Nubank Roxinho” com “ · vence dia 10”.
- Passo 4: vai para “Passo 4 de 8”.
- Passo 6: avança sem criar cartão nenhum.

Casos de erro a testar:

- “Dia do vencimento” `0` ou `32` → “Informe um dia entre 1 e 31”.
- Limite `-10,00` → “O limite não pode ser negativo”; limite vazio → “Informe o limite do cartão (use 0,00 se não souber)”.

- Nome vazio → “Informe o nome do cartão”.

□ Aprovado □ Reprovado — Observações: _____

Story 3.4 — Sugerir as categorias de gasto iniciais

Objetivo: mostrar 10 categorias sugeridas em 3 grupos, todas marcadas, e criar as que o usuário mantiver marcadas.

Pré-requisitos: usuário no passo 4.

Passos:

1. Confira o título “Categorias de gastos sugeridas” e os grupos: Essenciais (Moradia, Alimentação, Transporte, Saúde), Variáveis (Lazer, Educação, Vestuário, Assinaturas) e Financeiro (Investimentos, Reserva de Emergência). Cada uma tem ícone e cor e vem marcada.
2. Desmarque “Vestuário” e “Educação”.
3. Clique em “Próximo”.
4. No passo 5, clique em “Voltar”.
5. Clique em “Próximo” de novo.

Resultado esperado:

- Passo 3: vai para “Passo 5 de 8” e aparecem só as 8 categorias marcadas, sob os 3 grupos.
- Passo 4: o passo 4 abre só para leitura, com o texto “Suas categorias já foram criadas. Para renomear, trocar, excluir ou adicionar, toque em Próximo e personalize-as.” e os checkboxes desabilitados.
- Passo 5: volta ao passo 5 sem duplicar categorias.

Casos de erro a testar:

- Desmarcar todas → aparece “Escolha ao menos uma categoria” e o “Próximo” fica desabilitado.
- Backend parado ao clicar em “Próximo” → toast “Não foi possível salvar suas categorias. Verifique sua conexão e tente novamente.”
- Backend parado ao abrir o passo → botão “Tentar de novo” para recarregar as sugestões.

□ Aprovado □ Reprovado — Observações: _____

Story 3.5 — Personalizar as categorias sugeridas

Objetivo: renomear, trocar cor e ícone, excluir e adicionar categorias no passo 5, com salvamento imediato.

Pré-requisitos: usuário no passo 5, com as categorias criadas na Story 3.4.

Passos:

1. Confira o título “Personalize suas categorias”.
2. Clique no nome “Lazer”, troque para `Diversão` e pressione Enter (ou clique fora).
3. Clique no ícone de “Moradia”. No painel, escolha outra cor em “Cor” e outro ícone em “Ícone” e clique em “Fechar”.
4. No grupo “Variáveis”, clique em “Adicionar categoria”, digite `Pets` e clique em “Adicionar”.
5. Clique na lixeira de “Pets”. No modal ‘Excluir “Pets”?’, leia “A categoria sai do seu orçamento. Você pode criá-la de novo depois.” e clique em “Excluir”.

6. Recarregue a página e confira que as mudanças continuam lá.
7. Clique em “Próximo” (ou em “Pular personalização”).

Resultado esperado:

- Passos 2–5: cada mudança é salva sem botão de salvar e continua lá após o reload.
- O grupo só mostra a lixeira quando está vazio; ao excluir um grupo vazio, o modal diz “O grupo vazio sai da sua lista.”
- Passo 7: vai para “Passo 6 de 8”.

Casos de erro a testar:

- Renomear uma categoria para vazio → “Informe o nome da categoria”.
- Adicionar uma categoria com nome já existente (ex.: `Saúde`) → “Já existe uma categoria com esse nome”.
- Excluir categorias até sobrar uma e tentar excluir a última → “Mantenha ao menos uma categoria”.
- Em “Adicionar categoria”, deixar o nome vazio e clicar em “Adicionar” → “Informe o nome da categoria”; “Cancelar” (ou Esc) fecha o formulário.

□ Aprovado □ Reprovado — Observações: _____

Story 3.6 — Primeira importação de transações guiada

Objetivo: no passo 6, orientar como baixar o extrato e permitir importar agora (upload OFX/CSV) ou depois.

Pré-requisitos: usuário no passo 6, com ao menos uma conta; um arquivo OFX ou CSV de extrato real ou de teste.

Passos:

1. Confira o título “Importe suas transações” e o texto “Baixe o extrato do seu banco em formato OFX ou CSV e faça upload aqui”.
2. Em “Como baixar o extrato no seu banco”, abra os itens Nubank, Banco do Brasil, Itaú, Bradesco e Caixa e confira o passo a passo de cada um.
3. Escolha a conta de destino, clique em “Escolher arquivo”, selecione o OFX e clique em “Enviar arquivo”.
4. Acompanhe o status: “Arquivo enviado, processando…”, “Banco detectado: …”, “Classificando transações com IA… X de Y” e então “Pronta para revisão: …” (ou “Concluída: …”).
5. Se o status for “Aguardando revisão”, clique em “Revisar”, confirme a importação e volte para `/onboarding`.
6. Com a importação “Concluída”, clique em “Continuar”.
7. Com outro usuário no passo 6, clique em “Importar depois”.

Resultado esperado:

- Passo 4: o status é atualizado sozinho, sem recarregar a página.
- Passo 6: vai para “Passo 7 de 8” e a importação conta como a primeira concluída.
- Passo 7: avança para o passo 7 sem importar nada.

Casos de erro a testar:

- Enviar sem escolher o arquivo → “Escolha o arquivo do extrato”; sem a conta de destino → “Escolha a conta ou o cartão de destino”.
- Arquivo `.pdf` ou `.txt` → “Formato de arquivo não suportado”.
- Arquivo maior que 10 MB → “Arquivo muito grande (máximo 10MB)”.
- “Continuar” só aparece depois de uma importação “Concluída”; antes disso, só “Importar depois”.

□ Aprovado □ Reprovado — Observações: _____

Story 3.7 — Explicar a navegação em 3 camadas do Dashboard GPS

Objetivo: apresentar as 3 camadas do dashboard e a navegação por rolagem vertical, em 5 telas.

Pré-requisitos: usuário no passo 7.

Passos:

1. Confira “Passo 7 de 8” e a tela “Conheça o Dashboard GPS”, com as 3 camadas empilhadas e a legenda “Role verticalmente para navegar entre camadas”.
2. Clique em “Próximo” e passe pelas telas “Onde estou?”, “Para onde vou?”, “Preciso ajustar?” e “Pronto para usar o dashboard”.
3. Clique em “Voltar” em uma tela intermediária.
4. Na última tela, confira “Seu check-in semanal leva apenas 5 minutos” e clique em “Próximo”.
5. No passo 8, clique em “Voltar”.

Resultado esperado:

- Passo 2: cada tela tem a ilustração da camada.
- Passo 3: volta uma tela.
- Passo 4: vai para “Passo 8 de 8”.
- Passo 5: reabre o tour na última tela; ao clicar em “Próximo”, volta ao passo 8 sem repetir o tour.

Casos de erro a testar:

- “Voltar” na primeira tela do tour → volta ao passo 6 (importação), sem perder o progresso.
- Backend parado ao clicar em “Próximo” na última tela → toast “Não foi possível salvar seu progresso. Verifique sua conexão e tente novamente.”
- Com “reduzir movimento” ligado no sistema → as telas trocam sem animação.

□ Aprovado □ Reprovado — Observações: _____

Story 3.8 — Criar a meta inicial de Reserva de Emergência

Objetivo: criar a meta de Reserva de Emergência (ou pular) e concluir o onboarding com a mensagem de parabéns no dashboard.

Pré-requisitos: usuário no passo 8.

Passos:

1. Confira o título “Crie sua primeira meta: Reserva de Emergência” e os campos pré-preenchidos: “Nome da meta” = “Reserva de Emergência”, “Valor-alvo (R$)” = 25.000,00, “Data-alvo (opcional)” = hoje + 12 meses e “Aporte mensal (R$) (opcional)” com o valor calculado (≈ 2.083,34).
2. Mude o valor-alvo para `12.000,00` e confira que o aporte é recalculado (≈ 1.000,00) e que a prévia mostra 0% e “Previsão de conclusão: ”.
3. Edite o aporte manualmente e depois mude a data: o aporte não é mais sobrescrito.
4. Clique em “Criar meta e concluir”.
5. No dashboard, feche a mensagem pelo “X” e recarregue a página.
6. Abra `/onboarding` de novo.

7. Com outro usuário no passo 8, clique em “Pular por agora”.

Resultado esperado:

- Passo 4: redireciona para `/dashboard` com “Parabéns! Sua conta está configurada” e “Agora é só acompanhar seu dinheiro por aqui.”; a meta é gravada em `goals` e `onboarding_progress.is_completed = true`.
- Passo 5: a mensagem não volta após o reload.
- Passo 6: redireciona direto para `/dashboard`.
- Passo 7: mesma mensagem de parabéns; nenhuma meta é criada.

Casos de erro a testar:

- Valor-alvo `0` → “O valor-alvo precisa ser maior que zero”; vazio → “Informe o valor-alvo da meta”.
- Nome vazio → “Informe o nome da meta”.
- Aporte `-10,00` → “O aporte mensal não pode ser negativo”; sem aporte e sem data → “Informe um aporte mensal ou uma data-alvo para ver a previsão.”
- Meta criada mas a conclusão falha (backend parado logo após criar) → toast “Não foi possível concluir o onboarding. Verifique sua conexão e tente novamente.”; ao voltar, aparece “Sua meta já foi salva. Toque em Concluir para terminar.” e o botão “Concluir”, sem criar uma meta duplicada.

□ Aprovado □ Reprovado — Observações: _____

Epic 4 — Contas e cartões

Story 4.1 — Cadastrar várias contas bancárias

Objetivo: ver as contas ativas com o saldo total em `/contas` e cadastrar uma conta nova depois do onboarding.

Pré-requisitos: usuário logado com o onboarding concluído. Para testar o estado vazio, um usuário sem contas ativas (na prática, raro depois do onboarding, que cria ao menos “Minha Conta Principal”).

Passos:

1. No dashboard, na seção “Configuracoes da conta”, clicar em “Contas bancárias”.
2. Conferir o título “Contas bancárias”, o subtítulo “Onde seu dinheiro está hoje”, o bloco “Saldo total” e a lista “Suas contas (N)”.
3. Clicar em “Adicionar conta”.
4. No diálogo “Adicionar conta”, preencher “Nome da conta” = `Inter`, “Banco (opcional)” = escolher “Inter” na lista de sugestões, “Tipo da conta” = “Conta corrente”, “Saldo atual da conta (R$)” = `500,00`.
5. Clicar em “Salvar conta”.
6. Repetir com uma conta “Teste centavos” com saldo `0,10` e outra com `0,20`.
7. Clicar em “Voltar ao dashboard”.

Resultado esperado:

- Ao abrir o diálogo, o foco está no campo “Nome da conta”.
- Depois de salvar: o diálogo fecha, “Inter” aparece no fim da lista com “Conta corrente · Inter” e R$ 500,00, o “Saldo total” aumenta R$ 500,00 sem recarregar a página e aparece a faixa verde “Conta adicionada com sucesso”, que some depois de alguns segundos.
- As contas de 0,10 e 0,20 somam exatamente R$ 0,30 a mais no total (sem erro de arredondamento).
- Saldo negativo de uma conta e saldo total negativo aparecem em vermelho.
- Sem contas ativas: aparece “Você ainda não tem contas cadastradas” com o botão “Adicionar conta”.

Casos de erro a testar:

- “Nome da conta” vazio → erro “Informe o nome da conta” no campo; o diálogo continua aberto.
- Saldo vazio → “Informe o saldo inicial (use 0,00 se não souber)”.
- Saldo `12,345` ou `abc` → “Digite o saldo no formato 1.234,56 (até 2 casas depois da vírgula)”.
- Backend parado ao abrir `/contas` → “Não foi possível carregar suas contas. Verifique sua conexão e tente de novo.” com o botão “Tentar de novo”; com o backend parado ao salvar → “Não foi possível salvar a conta. Tente novamente.” dentro do diálogo, com os dados preservados.

□ Aprovado □ Reprovado — Observações: _____

Story 4.2 — Cadastrar vários cartões de crédito

Objetivo: ver os cartões ativos com o limite total em `/cartoes` e cadastrar um cartão novo.

Pré-requisitos: usuário logado com o onboarding concluído.

Passos:

1. No dashboard, clicar em “Cartões de crédito”.

2. Conferir o título “Cartões de crédito”, o bloco “Limite total” e a lista “Seus cartões (N)”.
3. Clicar em “Adicionar cartão”.
4. No diálogo “Adicionar cartão”, preencher “Nome do cartão” = `Inter Gold`, banco = `Inter`, “Limite do cartão (R$)” = `3.000,00`, “Dia do vencimento” = `15`.
5. Clicar em “Salvar cartão”.

Resultado esperado:

- O diálogo fecha, “Inter Gold” aparece na lista com “Inter · vence dia 15” e R$ 3.000,00, o “Limite total” aumenta R$ 3.000,00 e aparece “Cartão adicionado com sucesso”, que some depois de alguns segundos.
- Sem cartões: “Você ainda não tem cartões cadastrados” com o botão “Adicionar cartão”.
- A página não mostra crédito disponível (fica para o Epic 5).

Casos de erro a testar:

- “Dia do vencimento” = `0` ou `32` → “Informe um dia entre 1 e 31”.
- Limite `-10,00` → “O limite não pode ser negativo”; limite vazio → “Informe o limite do cartão (use 0,00 se não souber)”.
- Nome vazio → “Informe o nome do cartão”.
- Backend parado ao abrir → “Não foi possível carregar seus cartões. Verifique sua conexão e tente de novo.” com “Tentar de novo”; ao salvar → “Não foi possível salvar o cartão. Tente novamente.” no diálogo.

□ Aprovado □ Reprovado — Observações: _____

Story 4.3 — Editar informações de contas e cartões

Objetivo: corrigir nome, banco, tipo e saldo de uma conta, e nome, banco, limite e vencimento de um cartão.

Pré-requisitos: uma conta “Nubank” com saldo R$ 100,00 e um cartão com vencimento dia 10.

Passos:

1. Em `/contas`, clicar em “Editar” na conta “Nubank”.
2. Conferir que o diálogo “Editar conta” abre com os valores atuais preenchidos (saldo como `100,00`).
3. Mudar o nome para `Nubank PJ` e o saldo para `250,00`; clicar em “Salvar alterações”.
4. Editar de novo a mesma conta, mudando só o nome; salvar.
5. Em `/cartoes`, clicar em “Editar” no cartão; mudar o vencimento para `20` e o limite para `6.000,00`; clicar em “Salvar alterações”.

Resultado esperado:

- Passo 3: o diálogo fecha, a lista mostra “Nubank PJ” com R$ 250,00, o “Saldo total” sobe R$ 150,00, aparece “Informações atualizadas” e o foco volta ao botão “Editar” da conta.
- Em “Histórico” da conta (Story 4.7) aparece um ponto “Edição” com R$ 250,00; a edição só do nome (passo 4) não cria ponto novo.
- Passo 5: o cartão mostra “vence dia 20” e R$ 6.000,00; o “Limite total” é recalculado; aparece “Informações atualizadas”.
- O saldo inicial da conta (initial_balance) não muda: isso não é visível na UI, só no banco.

Casos de erro a testar:

- Na edição da conta, saldo vazio → “Informe o saldo atual (use 0,00 se não souber)”; nome vazio → “Informe o nome da conta”.

- Na edição do cartão, dia `0` → “Informe um dia entre 1 e 31”.
- Abrir “Editar” numa aba, remover o mesmo item em outra aba e salvar na primeira → o diálogo fecha e aparece o aviso âmbar “Esta conta não existe mais” (ou “Este cartão não existe mais”); o item some da lista.

□ Aprovado □ Reprovado — Observações: _____

Story 4.4 — Remover e restaurar contas e cartões

Objetivo: remover (soft delete) uma conta ou um cartão com confirmação e restaurá-lo depois.

Pré-requisitos: onboarding concluído; pelo menos 2 contas ativas e 1 cartão ativo.

Passos:

1. Em `/contas`, clicar em “Remover” numa conta.
2. Conferir o diálogo “Remover ?” com o texto “A conta some da lista, mas o histórico é preservado. Você pode restaurá-la depois.” Clicar em “Cancelar”.
3. Clicar de novo em “Remover” e confirmar com “Remover”.
4. Marcar a caixa “Mostrar removidas”.
5. Na seção “Removidas”, clicar em “Restaurar” na conta removida.
6. Repetir os passos 1 a 5 em `/cartoes` (caixa “Mostrar removidos”, seção “Removidos”).

Resultado esperado:

- Cancelar: nada muda.
- Confirmar: a conta some da lista, o “Saldo total” é recalculado e aparece “Conta removida”.
- “Removidas” lista a conta com o saldo e o botão “Restaurar”, sem “Editar”; o total não inclui contas removidas. Sem nenhuma removida: “Nenhuma conta removida”.
- Restaurar: a conta volta para a lista ativa e para o total, e aparece “Conta restaurada”. O histórico de saldo dela continua igual.
- Cartões: texto “O cartão some da lista, mas o histórico é preservado. Você pode restaurá-lo depois.”, mensagens “Cartão removido” e “Cartão restaurado”; sem removidos, “Nenhum cartão removido”.

Casos de erro a testar:

- Remover a última conta ativa → o diálogo continua aberto com “Mantenha ao menos uma conta ativa”.
- Remover o único cartão ativo → permitido (cartões não têm mínimo).
- Remover a mesma conta em duas abas e confirmar na segunda → aviso “Esta conta já foi removida ou não existe mais” (cartão: “Este cartão já foi removido ou não existe mais”).
- Restaurar em duas abas e clicar na segunda → “Esta conta já foi restaurada ou não existe mais” (cartão: “Este cartão já foi restaurado ou não existe mais”).

□ Aprovado □ Reprovado — Observações: _____

Story 4.5 — Registrar transferências entre contas próprias

Objetivo: mover dinheiro entre duas contas próprias, atualizando os dois saldos sem mudar o total.

Pré-requisitos: 2 contas ativas, por exemplo “Nubank” com R$ 100,00 e “Inter” com R$ 500,00.

Passos:

1. Em `/contas`, clicar em “Transferir”.
2. No diálogo “Nova transferência”, escolher “Conta de origem” = Nubank e “Conta de destino” = Inter (as opções mostram “Nome — R$ saldo”).
3. “Valor (R$)” = `150,00`; manter “Data” com o dia de hoje; “Descrição (opcional)” = `Reserva do mês`.
4. Clicar em “Transferir”.
5. Abrir o “Histórico” das duas contas.

Resultado esperado:

- O diálogo fecha e aparece “Transferência registrada”.
- Nubank fica com –R$ 50,00 em vermelho (saldo negativo é permitido), Inter com R$ 650,00; o “Saldo total” não muda.
- A seção “Transferências recentes” mostra “Nubank → Inter”, a data de hoje, “Reserva do mês” e R$ 150,00, com a mais recente primeiro. Sem transferências: “Nenhuma transferência registrada”.
- No histórico de cada conta aparece um ponto “Transferência”.
- Com só 1 conta ativa, o botão “Transferir” não aparece.

Casos de erro a testar:

- Mesma conta em origem e destino → “Escolha contas diferentes” no campo de destino.
- Valor `0` → “O valor precisa ser maior que zero”; valor `10,555` → “Digite o valor no formato 1.234,56 (até 2 casas depois da vírgula)”; vazio → “Informe o valor da transferência”.
- Data de amanhã (digitando, já que o calendário bloqueia datas futuras) → “A data não pode estar no futuro”.
- Remover a conta de destino em outra aba e transferir → o diálogo continua aberto com “Conta não encontrada” e a lista de contas é recarregada.

□ Aprovado □ Reprovado — Observações: _____

Story 4.6 — Reconciliar o saldo da conta com o banco

Objetivo: conferir o saldo do porquinho com o saldo do banco e ajustar a diferença.

Pré-requisitos: uma conta ativa com saldo R$ 1.000,00.

Passos:

1. Em `/contas`, clicar em “Reconciliar” na conta.
2. Conferir o diálogo “Reconciliar ” com “Saldo no porquinho: R$ 1.000,00”.
3. Em “Saldo no banco (R$)”, digitar `1.050,00`; conferir “Diferença: +R$ 50,00” e a pergunta “Deseja ajustar o saldo?”.
4. Preencher “Motivo (opcional)” = `Tarifa estornada` e clicar em “Ajustar saldo”.
5. Abrir “Reconciliar” de novo, digitar `1.050,00` (igual) e clicar em “Confirmar reconciliação”.
6. Abrir “Reconciliar” de novo, digitar `980,00` e clicar em “Investigar depois”.

Resultado esperado:

- A diferença atualiza enquanto se digita; negativa aparece em vermelho com sinal de menos; com o campo vazio aparece “—”.
- Passo 4: o diálogo fecha, a conta mostra R$ 1.050,00, aparece “Reconciliada em dd/mm/aaaa” (hoje), o “Saldo total” é recalculado e aparece “Saldo reconciliado”. No “Histórico” surge um ponto “Reconciliação”.
- Passo 5: com diferença zero, o botão é “Confirmar reconciliação” (com “Cancelar” ao lado); aparece “Saldo reconciliado” e nenhum ponto novo no histórico.

- Passo 6: o diálogo fecha e nada muda.
- Saldo do banco negativo (ex.: `-30,00`) é aceito.

Casos de erro a testar:

- "Saldo no banco" vazio e enviar → "Informe o saldo no banco".
- Valor `10,555` → "Digite o saldo no formato 1.234,56 (até 2 casas depois da vírgula)".
- Valor com mais de 13 dígitos antes da vírgula → "Informe o saldo com até 2 casas decimais e no máximo 13 dígitos antes da vírgula".
- Remover a conta em outra aba e reconciliar → o diálogo fecha, aparece o aviso "Esta conta não existe mais" e a lista é recarregada.

□ Aprovado □ Reprovado — Observações: _____

Story 4.7 — Ver o histórico do saldo ao longo do tempo

Objetivo: ver a evolução do saldo de uma conta num gráfico e numa tabela, por período.

Pré-requisitos: uma conta com eventos de criação, edição de saldo, transferência e reconciliação (feitos nas stories anteriores).

Passos:

1. Em `/contas`, clicar em "Histórico" na conta.
2. Conferir a URL `/contas/<id>/historico`, o título "Histórico de saldo — ", o "Saldo atual" e o link "Voltar para contas".
3. Clicar em cada período: "30 dias", "3 meses", "6 meses", "1 ano", "Tudo".
4. Passar o mouse num marcador do gráfico; depois usar Tab até o gráfico e as setas do teclado entre os pontos.
5. Conferir a tabela "Movimentações".

Resultado esperado:

- O período padrão é "3 meses"; o botão selecionado fica destacado.
- O gráfico tem eixo de datas, eixo em R$, um marcador por evento com cor/forma por tipo e legenda.
- Ao passar o mouse ou focar um ponto aparecem a data (dd/mm/aaaa), o saldo e o tipo: "Saldo inicial", "Edição", "Transferência", "Reconciliação" ou "Transação".
- A tabela tem as colunas "Data", "Evento", "Saldo", da mais recente para a mais antiga; saldos negativos em vermelho.
- Num período sem eventos, se houver um evento anterior, ele aparece como primeiro ponto; se não houver nada, aparece "Nenhuma movimentação neste período".

Casos de erro a testar:

- Copiar a URL do histórico, remover a conta e abrir a URL → o histórico continua aparecendo (conta removida não tem o botão "Histórico", mas a URL funciona).
- Abrir `/contas/<id-inexistente-ou-de-outro-usuário>/historico` (um UUID válido qualquer) → "Esta conta não existe mais." com o link "Voltar para contas".
- Backend parado → "Não foi possível carregar o histórico. Verifique sua conexão e tente de novo." com "Tentar de novo".
- Período inválido ("Período inválido"): não dá para testar pela UI, porque o período não fica na URL.

□ Aprovado □ Reprovado — Observações: _____

Story 4.R — Correções da retro do Epic 4 (edição concorrente de conta)

Objetivo: garantir que editar uma conta não desfaz uma transferência ou reconciliação feita em outra aba.

Pré-requisitos: 2 contas ativas; o app aberto em duas abas do navegador em `/contas`.

Passos:

1. Na aba A, clicar em “Editar” na conta “Nubank” e deixar o diálogo aberto.
2. Na aba B, fazer uma transferência de R$ 50,00 saindo de “Nubank”.
3. Na aba A, mudar o saldo para qualquer valor e clicar em “Salvar alterações”.
4. Repetir com uma reconciliação na aba B no lugar da transferência.
5. Na aba A, abrir “Editar” de novo e salvar.

Resultado esperado:

- Passo 3: o diálogo fecha, aparece o aviso âmbar “Esta conta foi alterada em outra tela. Abra a edição de novo.”, a lista é recarregada mostrando o saldo da transferência, e o foco volta ao “Editar” da conta. Nada é gravado: o histórico não ganha ponto “Edição”.
- Passo 4: o mesmo aviso.
- Passo 5: a edição é salva normalmente (“Informações atualizadas”).
- 404 na edição com “Mostrar removidas” marcada (conta removida na aba B): a conta sai da lista ativa e aparece em “Removidas”.
- A guarda contra respostas atrasadas (recarga lenta sobrescrevendo uma edição) não é verificável de forma confiável pela UI.

Casos de erro a testar:

- Duas edições da mesma conta em duas abas → a segunda a salvar recebe o aviso de alteração em outra tela.
- Cartões não têm esse controle: duas edições do mesmo cartão em duas abas → a última vence, sem aviso (comportamento esperado).

□ Aprovado □ Reprovado — Observações: _____

Epic 5 — Importação de transações e classificação por IA

Arquivos de exemplo: todos ficam em `porquinho-backend/src/test/resources/import-samples/` (dados fictícios, outubro de 2026). Abreviado abaixo como `import-samples/`.

- Conta: `nubank-conta.ofx` / `.csv`, `bb-conta.ofx` / `.csv`, `itau-conta.ofx` / `.csv`, `bradesco-conta.ofx` / `.csv`, `caixa-conta.ofx` / `.csv`
- Cartão: `itau-cartao.ofx`, `nubank-cartao.csv`
- Erro: `erro-ofx-truncado.ofx`, `erro-nao-e-extrato.ofx`, `erro-ofx-linhas-invalidas.ofx`, `erro-csv-banco-desconhecido.csv`, `erro-csv-linhas-invalidas.csv`, `erro-arquivo-vazio.csv`

Arquivos montados à mão: prefira CSV no layout Nubank (UTF-8, fácil de editar). Conta: cabeçalho `Data,Valor,Identificador,Descrição`, data `DD/MM/AAAA`, valor com ponto e sinal (débito negativo), identificador único por linha. Cartão: cabeçalho `date,title,amount`, data `AAAA-MM-DD`, compra positiva. Os OFX estão em cp1252: se editar um, salve na mesma codificação.

Pré-requisitos comuns: usuário logado com assinatura ou trial ativo (sem isso a API responde 402), pelo menos uma conta (`/contas`) e um cartão (`/cartoes`) cadastrados. O caminho até a importação é o link “Importar extrato” do dashboard (`/importar`).

IA: a classificação automática (Story 5.8) só funciona com `ANTHROPIC_API_KEY` definida no backend. Sem a chave, nenhuma chamada é feita e toda transação importada chega à revisão como “Não classificado” (grupo “Baixa confiança”). As stories 5.9 a 5.13 funcionam sem a chave.

Story 5.1 — Upload de arquivos OFX dos 5 bancos

Objetivo: enviar um extrato OFX para uma conta ou um cartão escolhido antes do envio.

Pré-requisitos: uma conta e um cartão cadastrados.

Passos:

1. Acesse `/importar` (título “Importar extrato”).
2. Em “Destino”, escolha uma conta no grupo “Contas”.
3. Clique em “Escolher arquivo” e selecione `import-samples/nubank-conta.ofx` (ou arraste o arquivo para a área tracejada).
4. Confira o nome e o tamanho do arquivo abaixo do botão.
5. Clique em “Enviar arquivo”.
6. Repita com destino = um cartão (grupo “Cartões de crédito”) e `import-samples/itau-cartao.ofx`.

Resultado esperado:

- Durante o envio aparece a barra “Enviando… N%” e o botão muda para “Enviando…”.
- Depois: “Arquivo enviado, processando…” em verde.
- A importação entra no topo de “Importações recentes” com nome do arquivo, data/hora e “Conta: ” ou “Cartão: ”.
- Sem conta nem cartão cadastrado, a página mostra “Para importar um extrato, cadastre antes a conta ou o cartão de destino.” com os links “Cadastrar conta” e “Cadastrar cartão”.

Casos de erro a testar:

- Clicar em “Enviar arquivo” sem destino → “Escolha a conta ou o cartão de destino”; com destino e sem arquivo → “Escolha o arquivo do extrato”.
- `itau-cartao.ofx` com destino = conta → “Este arquivo é de cartão de crédito. Escolha um cartão como destino.”; `nubank-conta.ofx` com destino = cartão → “Este arquivo é de conta bancária. Escolha uma conta como destino.”
- Arquivo `.pdf` ou `.txt` (ou `erro-arquivo-vazio.csv`) → “Formato de arquivo não suportado”, sem envio. `erro-nao-e-extrato.ofx` → mesma mensagem, vinda do servidor.
- Arquivo com mais de 10MB (ex.: `mkfile 11m grande.ofx`) → “Arquivo muito grande (máximo 10MB)” antes do envio.

□ Aprovado □ Reprovado — Observações: _____

Story 5.2 — Upload de arquivos CSV dos 5 bancos

Objetivo: importar extratos CSV, com o layout reconhecendo o banco e o tipo (conta ou cartão).

Pré-requisitos: uma conta e um cartão cadastrados. Use contas diferentes para cada banco, ou descarte a importação anterior, para não cair em duplicatas (Story 5.5).

Passos:

1. Em `/importar`, destino = uma conta, envie `import-samples/bb-conta.csv`.
2. Aguarde o status na lista mudar sozinho (consulta a cada 2 s).
3. Repita com `itau-conta.csv`, `bradesco-conta.csv`, `caixa-conta.csv` e `nubank-conta.csv` (cada um numa conta nova ou após descartar o anterior).
4. Destino = um cartão, envie `import-samples/nubank-cartao.csv`.
5. Abra “Revisar” de `bb-conta.csv` e confira os acentos nas descrições.

Resultado esperado:

- Cada importação termina em “Pronta para revisão: N transações”, com o mesmo N do OFX do mesmo banco (BB 10, Itaú 10, Bradesco 9, Caixa 10, Nubank conta 10; Nubank cartão 9).
- Linhas de saldo, total, título e rodapé (BB e Bradesco) não viram transações e não aparecem como “linhas com erro”.
- No cartão Nubank, “Pagamento recebido” e “Estorno - Amazon Marketplace” aparecem com valor positivo (crédito); as compras, negativas.
- Acentos corretos (ex.: “Transferência”, “Itaú”).

Casos de erro a testar:

- `erro-csv-banco-desconhecido.csv` → “Não reconhecemos o formato deste CSV. Tente exportar o extrato em OFX.” e nada entra na lista.
- `nubank-cartao.csv` com destino = conta → “Este arquivo é de cartão de crédito. Escolha um cartão como destino.”
- `nubank-conta.csv` com destino = cartão → “Este arquivo é de conta bancária. Escolha uma conta como destino.”
- `erro-arquivo-vazio.csv` → “Formato de arquivo não suportado”.

□ Aprovado □ Reprovado — Observações: _____

Story 5.3 — Detecção automática de formato e banco

Objetivo: o sistema identifica o banco do OFX em segundo plano e mostra o andamento na lista.

Pré-requisitos: uma conta cadastrada.

Passos:

1. Em `/importar`, destino = conta, envie `import-samples/caixa-conta.ofx`.
2. Observe a linha da importação em "Importações recentes" logo após o envio.
3. Monte um OFX de banco desconhecido: copie `caixa-conta.ofx` para `banco-teste.ofx` e troque `<ORG>CAIXA ECONOMICA FEDERAL</ORG>` por `<ORG>BANCO TESTE</ORG>`, `<FID>104</FID>` por `<FID>077</FID>` e `<BANKID>104</BANKID>` por `<BANKID>077</BANKID>`. Envie para outra conta.
4. Envie `import-samples/erro-ofx-truncado.ofx`.

Resultado esperado:

- Logo após o envio: "Recebido — identificando o banco…" e depois "Banco detectado: Caixa — lendo as transações…" (pode passar rápido; recarregue a página logo após enviar para ver).
- Para `banco-teste.ofx`: "Banco não identificado — lendo as transações…" e a importação segue normalmente até "Pronta para revisão".
- A lista para de consultar quando nenhuma importação está em processamento.

Casos de erro a testar:

- `erro-ofx-truncado.ofx` → status "Falhou" e, em vermelho, "O arquivo está incompleto. Exporte o extrato de novo no site ou app do banco e envie outra vez."
- Reiniciar o backend logo após um envio (importação ainda em processamento) → na subida a importação vai para "Falhou" com "O processamento foi interrompido. Envie o arquivo de novo." (ou para "Pronta para revisão", se as linhas já tinham sido gravadas).
- Abrir `/importar/<id-inexistente>/revisao` → "Esta importação não existe mais."

□ Aprovado □ Reprovado — Observações: _____

Story 5.4 — Parsing de transações com tratamento robusto de erros

Objetivo: ler os lançamentos válidos, pular e contar as linhas com problema.

Pré-requisitos: uma conta cadastrada (sem importações anteriores dos mesmos arquivos).

Passos:

1. Em `/importar`, destino = conta, envie `import-samples/erro-ofx-linhas-invalidas.ofx` (OFX Itaú com 2 lançamentos bons e 4 ruins: data inválida, `TRNAMT` ausente, `TRNAMT` "abc" e "0.00").
2. Em outra conta (ou após descartar), envie `import-samples/erro-csv-linhas-invalidas.csv`.
3. Clique em "Revisar" e confira as 2 transações de cada arquivo.
4. Para um teste próprio: copie `nubank-conta.csv`, troque uma data por `32/10/2026`, um valor por `abc` e apague o valor de outra linha; troque também os identificadores de todas as linhas (para não virarem duplicatas) e envie.

Resultado esperado:

- Passos 1 e 2: "Pronta para revisão: 2 transações (4 linhas com erro)".

- Passo 4: “Pronta para revisão: 7 transações (3 linhas com erro)”.
- No OFX, o lançamento com vírgula decimal e acento aparece com valor e acentos corretos.
- No `itau-conta.ofx` / `nubank-conta.ofx`, o pagamento de fatura aparece no grupo “Pagamento de fatura” na revisão.

Casos de erro a testar:

- OFX completo sem nenhum `<STMTTRN>` (copie `caixa-conta.ofx` e apague todos os blocos `<STMTTRN>…</STMTTRN>`) → “Falhou” com “Não encontramos transações neste arquivo. Exporte o extrato de novo e envie outra vez.”
- `erro-ofx-truncado.ofx` → “Falhou” com a mensagem de arquivo incompleto.
- CSV em que todas as linhas são inválidas → importação não chega a “Pronta para revisão” com transações (anote o comportamento).

□ Aprovado □ Reprovado — Observações: _____

Story 5.5 — Detectar e descartar transações duplicadas

Objetivo: não gravar de novo lançamentos que já estão no destino, nem transferências já registradas.

Pré-requisitos: uma conta “Nubank” cadastrada. Para o caso de transferência: contas “BB” e “Nubank” e uma transferência registrada em `/contas` (Story 4.5) de BB para Nubank, R$ 2.000,00, em 01/10/2026.

Passos:

1. Destino = conta Nubank, envie `import-samples/nubank-conta.ofx`. Aguarde “Pronta para revisão”.
2. Envie o mesmo arquivo de novo para a mesma conta.
3. Envie `import-samples/nubank-conta.csv` para a mesma conta (mesmo mês, outro formato).
4. Reimportação parcial: copie `nubank-conta.csv` para `parcial.csv`, mantenha o cabeçalho e as 3 primeiras linhas e acrescente `28/10/2026,-42.00,novo-id-0001,Compra no débito - PADARIA TESTE`. Envie para a mesma conta.
5. Transferência: destino = conta BB, envie `import-samples/bb-conta.ofx`.

Resultado esperado:

- Passo 2: status “Nenhuma transação nova — 10 duplicatas detectadas e ignoradas” (sem link “Revisar”).
- Passo 3: mesmo resultado (todas as linhas viram duplicatas, mesmo em outro formato).
- Passo 4: “Pronta para revisão: 1 transação · 3 duplicatas detectadas e ignoradas”; na revisão só aparece “PADARIA TESTE”.
- Passo 5: o “Pix - Enviado” de R$ 2.000,00 não é importado e o status inclui “· 1 transferência já registrada”.

Casos de erro a testar:

- Arquivo com duas linhas idênticas (mesma data, valor e descrição, identificadores diferentes) e nada no banco → as 2 são gravadas.
- Mesmo arquivo para uma conta diferente → nenhuma duplicata (a checagem é por destino).
- Descartar a primeira importação (“Descartar importação” na revisão) e reenviar o mesmo arquivo → importa tudo de novo, sem duplicatas.
- Na importação para cartão, a checagem de transferências não se aplica: nenhum “transferência já registrada”.

□ Aprovado □ Reprovado — Observações: _____

Story 5.6 — Identificar Pix e extrair detalhes

Objetivo: marcar os lançamentos Pix e mostrar para quem foi ou de quem veio.

Pré-requisitos: uma conta cadastrada, sem importações anteriores destes arquivos.

Passos:

1. Destino = conta, envie `import-samples/nubank-conta.csv` e clique em "Revisar".
2. Procure as linhas "Transferência recebida pelo Pix - JOANA PEREIRA LIMA …" e "Transferência enviada pelo Pix - MERCADO BOM PRECO LTDA …".
3. Monte `pix-teste.csv` (layout Nubank conta) com as linhas:
   - `03/10/2026,-30.00,pix-0001,PIX TRANSF CARLOS 03/10`
   - `04/10/2026,-25.00,pix-0002,TRANSFERENCIA PIX DES: CARLOS EDUARDO SOUZA 04/10`
   - `05/10/2026,-10.00,pix-0003,ENVIO PIX`
   - `06/10/2026,15.00,pix-0004,ESTORNO PIX LOJA X`
   - `07/10/2026,-99.00,pix-0005,PIXEL STORE`
4. Envie para outra conta e abra "Revisar".

Resultado esperado:

- Pix aparece sempre com ícone e texto: "Pix recebido de JOANA PEREIRA LIMA", "Pix enviado para MERCADO BOM PRECO LTDA".
- `pix-teste.csv`: "Pix enviado para CARLOS", "Pix enviado para CARLOS EDUARDO SOUZA", "Pix enviado" (sem nome), "Pix recebido" (estorno, sem nome).
- "PIXEL STORE" não tem marcador de Pix.
- Após concluir a importação, os mesmos textos aparecem em `/transacoes`.

Casos de erro a testar:

- Descrição só com "pix" no meio ("Compra pix loja") → marcada como Pix, sem nome.
- Descrição com "PIXEL" ou "PIXBET" (sem a palavra isolada) → não é Pix.
- CPF completo na descrição (`PIX 123.456.789-09`) → aparece mascarado no detalhe salvo (confira se o texto exibido não expõe o CPF inteiro e anote).

□ Aprovado □ Reprovado — Observações: _____

Story 5.7 — Detectar compras parceladas e criar parcelas futuras

Objetivo: reconhecer o marcador de parcela nas compras de cartão e criar as parcelas que faltam.

Pré-requisitos: um cartão cadastrado. Marcadores reconhecidos: "Parcela 3/10", "PARC 03/10" (ou "PARC. 03/10") e um "03/10" no fim da descrição precedido de espaço. Vale só com 1 ≤ número ≤ total e 2 ≤ total ≤ 48, e só em importações para cartão.

Passos:

1. Destino = cartão, envie `import-samples/nubank-cartao.csv` e abra "Revisar".
2. Confira "Magazine Luiza - Parcela 3/10" e "Smart Fit - Parcela 2/12".
3. Em outro cartão, envie `import-samples/itau-cartao.ofx` e confira "MAGAZINELUIZA 03/10", "CASASBAHIA 07/12", "MERCADOLIVRE*LOJA 01/03" e "ANUIDADE DIFERENCIADA 05/12".
4. Monte `parcelas.csv` (layout Nubank cartão) com:

- `2026-10-03,Loja Teste PARC 01/03,100.00`
- `2026-10-04,Loja Final 03/03,50.00`
- `2026-10-05,Loja Invalida 15/10,20.00`
- `2026-10-06,Loja Zero 0/10,20.00`

5. Envie para um cartão e abra “Revisar”.

Resultado esperado:

- “Compra parcelada: 3/10 — restam 7 parcelas” (Magazine Luiza) e “Compra parcelada: 2/12 — restam 10 parcelas” (Smart Fit).
- Itaú: “Compra parcelada: 7/12 — restam 5 parcelas” na Casas Bahia; “Compra parcelada: 1/3 — restam 2 parcelas” no Mercado Livre.
- `parcelas.csv`: “Compra parcelada: 1/3 — restam 2 parcelas” e “Compra parcelada: 3/3 — última parcela”; as linhas 15/10 e 0/10 não têm marcador.
- As parcelas futuras não aparecem na revisão nem na lista de `/transacoes`: só a contagem “restam N”.

Casos de erro a testar:

- Conta (não cartão) com descrição “PIX TRANSF CARLOS 03/10” → não vira parcela.
- Data no fim (“LOJA 03/10/26”) → não é parcela.
- Mês seguinte: monte um CSV de cartão com `2026-11-02,Magazine Luiza - Parcela 4/10,189.90` e envie para o mesmo cartão do passo 1 (com a importação do passo 1 concluída) → a parcela futura 4/10 é confirmada; a revisão mostra “Compra parcelada: 4/10 — restam 6 parcelas” e nenhuma parcela nova é criada.
- Reenviar `nubank-cartao.csv` no mesmo cartão → duplicatas, nenhuma parcela futura nova.

□ Aprovado □ Reprovado — Observações: _____

Story 5.8 — Classificação automática com IA

Objetivo: a IA sugere categoria e confiança para cada transação importada, sem bloquear a importação quando está indisponível.

Pré-requisitos: categorias criadas (onboarding ou `/categorias`). Para o teste com IA: backend iniciado com `ANTHROPIC_API_KEY` válida. Para o teste sem IA: backend sem a variável.

Passos:

1. Com a chave configurada, destino = conta, envie `import-samples/itau-conta.ofx`.
2. Observe a linha da importação enquanto processa.
3. Clique em “Revisar” quando aparecer “Pronta para revisão”.
4. Reinicie o backend sem `ANTHROPIC_API_KEY` e envie `import-samples/bradesco-conta.ofx` para outra conta.
5. Abra “Revisar”.

Resultado esperado:

- Com chave: durante o processamento aparece “Classificando transações com IA… X de Y”, atualizado a cada 2 s.
- Na revisão, cada transação tem categoria no formato “Grupo › Categoria” e confiança, ex.: “Alta confiança (92%)”.
- Pagamentos de fatura não passam pela IA e ficam no grupo “Pagamento de fatura”.

- Sem chave: a importação chega a “Pronta para revisão” normalmente; todas as transações ficam no grupo “Baixa confiança” com “Não classificado”.

Casos de erro a testar:

- Chave inválida (ex.: `ANTHROPIC_API_KEY=xxx`) → importação não falha; transações chegam “Não classificado”.
- Importação só com duplicatas → “Nenhuma transação nova — …”, sem passar por “Classificando”.
- Parcela com futuras (`nubank-cartao.csv`) → as futuras herdam a categoria da parcela (confira em `/transacoes` nos meses seguintes após concluir).

□ Aprovado □ Reprovado — Observações: _____

Story 5.9 — Apresentar as transações classificadas para revisão

Objetivo: mostrar as transações agrupadas por confiança, com Pix, parcela e aprovação em lote.

Pré-requisitos: uma importação em “Aguardando revisão”/“Pronta para revisão” (ex.: `nubank-conta.ofx` numa conta). Para ver os grupos Alta/Média, IA ativa (5.8) ou padrões aprendidos (5.11).

Passos:

1. Em `/importar`, clique em “Revisar” na importação.
2. Confira o título “Vamos revisar”, a linha “ · ” e a contagem.
3. Confira os grupos e os marcadores de cada linha.
4. Clique em “Aprovar todas de alta confiança”.
5. Clique em “Continuar depois”.
6. Volte pelo “Revisar” e clique em “Descartar importação” → “Descartar”.

Resultado esperado:

- Contagem no formato “X transações de alta confiança, Y precisam de revisão” (ex.: “0 transações de alta confiança, 9 precisam de revisão” sem IA no `nubank-conta.ofx`).
- Grupos, na ordem e só quando têm linhas: “Alta confiança (N)”, “Média confiança (N)”, “Baixa confiança (N)”, “Revisadas por você (N)”, “Pagamento de fatura (N)”.
- Indicadores com ícone e texto: “Alta confiança (92%)” (verde), “Média confiança (60%)” (amarelo), “Baixa confiança (30%)” ou “Não classificado” (vermelho). Limites: 80 = Alta, 79 e 50 = Média, 49 = Baixa.
- Passo 4: “N transações aprovadas” e as linhas ganham “Aprovada”. Sem alta confiança pendente, o botão fica desabilitado.
- Passo 5: volta para `/importar`, importação ainda com “Revisar”.
- Passo 6: diálogo “Descartar importação” com “As transações deste arquivo serão removidas e nada entra nas suas contas. Você pode enviar o mesmo arquivo de novo depois.”; após descartar, volta para `/importar` com “Descartada — nenhuma transação foi importada”.

Casos de erro a testar:

- Abrir a URL de revisão de uma importação concluída ou descartada → “Esta importação não está em revisão.”
- Abrir `/importar/<id de outro usuário>/revisao` (logado com outro usuário) → “Esta importação não existe mais.”
- Importação A com duplicatas em B (B enviada depois, mesmo destino): ao descartar A, o diálogo avisa “Arquivos enviados depois deste tiveram lançamentos ignorados como duplicatas: **. Depois de descartar, envie esses arquivos de novo para recuperar os lançamentos.”**

- Importação só de pagamento de fatura → “0 transações de alta confiança, 0 precisam de revisão” e “Aprovar todas de alta confiança” desabilitado.

□ Aprovado □ Reprovado — Observações: _____

Story 5.10 — Revisar e ajustar classificações e concluir a importação

Objetivo: corrigir categorias (uma ou várias) e concluir a importação, aplicando o efeito no saldo.

Pré-requisitos: uma importação em revisão numa conta (ex.: `nubank-conta.ofx`) e outra num cartão (ex.: `nubank-cartao.csv`). Anote o saldo da conta em `/contas` antes. Atenção: só transações com data **posterior** ao último ajuste de saldo (criação da conta, edição ou reconciliação) alteram o saldo; as amostras são de outubro de 2026, então numa conta criada depois disso o saldo não muda.

Passos:

1. Na revisão, clique em “Editar” numa linha. No diálogo “Editar categoria”, confira Data, Descrição, Valor e “Categoria atual”; escolha em “Nova categoria” e clique em “Salvar”.
2. Marque 2 linhas pela caixa de seleção e clique em “Alterar categoria (2)”; escolha a categoria e clique em “Salvar”.
3. Na importação do cartão, edite “Magazine Luiza - Parcela 3/10”.
4. Clique em “Concluir importação”, leia o resumo e clique em “Concluir”.
5. Confira o saldo da conta em `/contas`.

Resultado esperado:

- Passo 1: “1 categoria alterada”; a linha vai para “Revisadas por você” com “Revisada por você”; Y da contagem diminui 1.
- Passo 2: “2 categorias alteradas”.
- Passo 3: o diálogo avisa “Aqui você troca a categoria desta transação. As parcelas futuras desta compra recebem a mesma categoria.”
- Passo 4: o diálogo “Concluir importação” mostra “N transações seguem sem revisão”, “N transações seguem sem categoria” e “Depois de concluir, não dá para desfazer.”; para conta, “Só as transações posteriores ao último ajuste do saldo alteram o saldo da conta.”; para cartão, “As transações entram no cartão e não alteram o saldo de nenhuma conta.”
- Após concluir, volta para `/importar` com “Concluída: N transações”.
- Saldo: só muda se houver transações posteriores ao último ajuste; cartão nunca muda saldo.

Casos de erro a testar:

- Abrir a revisão de uma importação já concluída → “Esta importação não está em revisão.”
- Remover a conta de destino (em `/contas`) com a importação em revisão e clicar em “Concluir” → “A conta desta importação foi removida: restaure ou descarte a importação”.
- O seletor “Nova categoria” lista só categorias dentro dos grupos (grupos não são selecionáveis).
- Onboarding no passo 6: importar e concluir por ali → volta para `/onboarding` (opcional, se houver usuário novo).

□ Aprovado □ Reprovado — Observações: _____

Story 5.11 — Aprender com as correções do usuário

Objetivo: uma correção na revisão vira padrão e classifica a mesma descrição nas próximas importações, sem IA.

Pré-requisitos: uma conta cadastrada; funciona com ou sem `ANTHROPIC_API_KEY`.

Passos:

1. Destino = conta, envie `import-samples/nubank-conta.ofx` e abra "Revisar".
2. Edite "Compra no débito - UBER *TRIP" para uma categoria (ex.: Transporte) e clique em "Salvar".
3. Clique em "Descartar importação" → "Descartar" (o padrão não é desfeito).
4. Envie de novo `nubank-conta.ofx` para a mesma conta e abra "Revisar".
5. Monte um CSV (layout Nubank conta) com `20/10/2026,-31.50,uber-0002,Compra no débito - UBER *TRIP 20/10` e envie para a mesma conta.

Resultado esperado:

- Passo 4: "Compra no débito - UBER *TRIP" aparece em "Alta confiança" com "Alta confiança (100%)" e a categoria escolhida, sem precisar da IA. A contagem passa a ter "1 transação de alta confiança".
- Passo 5: a nova linha também vem classificada com a mesma categoria (100%), pois a descrição normalizada ignora datas, números, acentos e símbolos. Uma descrição só parecida (ex.: "Uber Trip", sem o "Compra no débito") não casa: o padrão é a descrição inteira normalizada.

Casos de erro a testar:

- Corrigir uma linha cuja descrição é só número ("123456") → não aprende; na próxima importação vai para a IA ou fica "Não classificado".
- Corrigir "PIX 98765" → não aprende (só termos bancários genéricos).
- Apagar a categoria usada no padrão (no onboarding/edição de categorias) e reimportar → a linha volta a ir para a IA ("Não classificado" sem chave).
- Logar com outro usuário e importar o mesmo arquivo → não usa o padrão do primeiro.

☐ Aprovado ☐ Reprovado — Observações: _____

Story 5.12 — Editar manualmente qualquer transação

Objetivo: ver, corrigir e remover transações confirmadas, e remover parcelas futuras de uma compra.

Pré-requisitos: importações concluídas de `nubank-conta.ofx` (conta) e `nubank-cartao.csv` (cartão). As amostras são de outubro de 2026.

Passos:

1. No dashboard, clique em "Transações" (`/transacoes`). A tela abre no mês atual; clique em "‹ Mês anterior" até outubro de 2026.
2. Confira cada linha: data, descrição, valor, conta/cartão, categoria, Pix e parcela.
3. Clique em "Editar" numa transação de conta; mude o valor e a categoria e clique em "Salvar".
4. Clique em "Remover" numa transação e confirme em "Remover".
5. Na linha "Magazine Luiza - Parcela 3/10", clique em "Remover parcelas futuras desta compra" e confirme em "Remover".

Resultado esperado:

- Pagamento de fatura mostra “Pagamento de fatura” no lugar da categoria; sem categoria, “Não classificado”.
- Passo 3: “Transação atualizada”. Se a data for posterior ao último ajuste de saldo, o saldo da conta muda pela diferença.
- Passo 4: o diálogo diz “"" sai da lista e, se tiver alterado o saldo da conta, o saldo volta ao que era.”; depois, “Transação removida”.
- Passo 5: o diálogo diz “7 parcelas futuras desta compra serão removidas, e as próximas importações não vão criá-las de novo. Use quando a compra foi estornada ou quitada.”; depois, “7 parcelas futuras removidas” e o rótulo fica “Compra parcelada: 3/10” (sem “restam”).
- Linhas de importação ainda em revisão não aparecem na lista.

Casos de erro a testar:

- Valor 0 ou vazio → “Informe um valor maior que zero”; descrição vazia → “Informe a descrição”.
- Editar uma parcela → os campos “Tipo” e “Conta ou cartão” ficam bloqueados com “Numa parcela, a conta ou o cartão e o tipo não mudam.”
- Data futura → aviso “Esta data ainda não chegou. Confira se é isso mesmo.” (não bloqueia).
- Remover a única parcela confirmada de uma compra → o diálogo avisa “É a única parcela confirmada desta compra: N parcelas futuras também serão removidas.”

□ Aprovado □ Reprovado — Observações: _____

Story 5.13 — Adicionar transações manualmente (despesas em dinheiro)

Objetivo: registrar um gasto em dinheiro ou um lançamento que nenhum extrato trouxe.

Pré-requisitos: uma conta (de preferência do tipo Carteira) e categorias criadas. Anote o saldo da conta.

Passos:

1. Em `/transacoes`, clique em “Adicionar transação”.
2. Confira o diálogo “Adicionar transação” com “Registre um gasto em dinheiro ou um lançamento que nenhum extrato trouxe.”
3. Preencha Data = amanhã, Descrição = “Feira”, Valor (R$) = 50,00, Tipo = “Saída (débito)”, Categoria e “Conta ou cartão” = a conta; clique em “Salvar”.
4. Confira o saldo em `/contas`.
5. Adicione outra com Tipo = “Entrada (crédito)”, valor 30,00, num cartão.

Resultado esperado:

- “Transação adicionada” e a lista recarrega com a nova linha no mês da data.
- Com data posterior ao último ajuste de saldo, o saldo da conta diminui R$ 50,00. Data de hoje ou anterior ao ajuste não altera o saldo.
- Lançamento no cartão não altera o saldo de nenhuma conta.

Casos de erro a testar:

- Salvar sem categoria → “Escolha uma categoria”; sem conta/cartão → “Escolha uma conta ou um cartão”; sem data → “Informe a data”.
- Valor 0 → “Informe um valor maior que zero”; valor com mais de 13 dígitos antes da vírgula → “Informe um valor de no máximo 13 dígitos antes da vírgula”.
- Usuário sem contas → aviso “Para gastos em dinheiro, cadastre uma conta Carteira em Contas.”
- O seletor de categoria não permite escolher um grupo (só categorias dentro dos grupos).

□ Aprovado □ Reprovado — Observações: _____

Epic 6 — Orçamento e categorias (método dos envelopes)

Story 6.1 — Criar categorias de gasto personalizadas

Objetivo: criar categorias próprias, com cor e ícone opcionais, fora do onboarding.

Pré-requisitos: usuário com grupos de categorias (vindos do onboarding).

Passos:

1. No dashboard, clique em "Categorias" (`/categorias`).
2. Confira o título "Categorias", o subtítulo "Os envelopes do seu orçamento, organizados por grupo" e a lista por grupo.
3. Clique em "Adicionar categoria". Confira que "Grupo" já vem com o primeiro grupo (ordem alfabética) e a descrição "Cor e ícone são opcionais: sem eles, a categoria usa a cor do grupo e o ícone de pasta."
4. Digite "Pets" em "Nome", deixe "Cor" em "Automática" e "Ícone" em "Padrão"; clique em "Salvar".
5. Crie outra ("Academia") escolhendo uma cor e um ícone.
6. Abra `/importar/<id>/revisao` ou "Adicionar transação" em `/transacoes` e confira o seletor de categoria.

Resultado esperado:

- "Categoria criada"; "Pets" aparece no grupo com a cor do grupo e ícone de pasta.
- "Academia" aparece com a cor e o ícone escolhidos.
- As novas categorias aparecem nos seletores de classificação.

Casos de erro a testar:

- Nome vazio ou só espaços → "Informe o nome da categoria" no campo, sem envio.
- Nome repetido sem diferenciar maiúsculas ("pets" com "Pets" existente) → "Já existe uma categoria com esse nome" no campo; o diálogo continua aberto.
- Nome com mais de 50 caracteres → o campo não aceita além de 50.
- Falha de rede ao salvar (desligue o backend) → "Não foi possível criar a categoria. Verifique sua conexão e tente novamente."

□ Aprovado □ Reprovado — Observações: _____

Story 6.2 — Organizar categorias em grupos

Objetivo: criar grupos, mover categorias entre grupos e recolher/expandir grupos (hierarquia de 1 nível).

Pré-requisitos: pelo menos dois grupos com categorias em `/categorias` .

Passos:

1. Em `/categorias` , clique em "Adicionar categoria", escolha "Nenhum (criar como grupo)" em "Grupo". O título vira "Adicionar grupo" e a descrição "Cor e ícone são opcionais: sem eles, o grupo recebe uma

cor livre da paleta e o ícone de pasta.”

2. Digite “Casa” e clique em “Salvar”.
3. Clique em “Mover” numa categoria que usa a cor do grupo. No diálogo “Mover ” (“Escolha o grupo de destino.”), escolha “Casa” e clique em “Salvar”.
4. Mova uma categoria com cor escolhida à mão (ex.: “Academia” da 6.1) para outro grupo.
5. Clique no nome de um grupo para recolher; clique de novo para expandir.

Resultado esperado:

- Passo 2: “Grupo criado”; “Casa” aparece com “Nenhuma categoria neste grupo”, cor livre da paleta e ícone de pasta.
- Passo 3: “Categoria movida para Casa”; a categoria aparece em “Casa” com a cor de “Casa”.
- Passo 4: a categoria muda de grupo e mantém a cor própria.
- Grupo que fica vazio após mover continua na lista com “Nenhuma categoria neste grupo”.
- Passo 5: as categorias do grupo somem e voltam; o grupo começa expandido ao abrir a página.
- O seletor do diálogo “Mover” não lista o grupo atual da categoria.

Casos de erro a testar:

- Grupo com nome repetido (“casa” com “Casa” existente) → “Já existe uma categoria com esse nome”.
- Usuário com um só grupo → botão “Mover” desabilitado.
- Grupos não têm botão “Mover” (não podem entrar em outro grupo) e não aparecem como opção nos seletores de categoria de transação.
- Falha de rede ao mover → “Não foi possível mover a categoria. Verifique sua conexão e tente novamente.”

□ Aprovado □ Reprovado — Observações: _____

Apêndice: divergências entre specs e código

Estes pontos apareceram ao montar o roteiro a partir da leitura do código. Ninguém os testou no navegador ainda: confirme cada um durante o teste.

Prováveis bugs (confirmar)

- 1.2, cadastro com e-mail e senha: o `RegisterView` não chama `authStore.setUser`. Depois do cadastro, a proteção de rotas manda para `/login` em vez de `/onboarding`.
- 1.4, limite de 3 pedidos por hora: o frontend envia o e-mail de redefinição pelo Supabase antes de chamar a auditoria do backend e ignora o 429. Por isso a mensagem de limite não aparece.
- 1.7, login depois de excluir a conta: o Supabase autentica e o backend responde 404. O frontend tenta recriar o usuário, recebe 409, registra só um `console.warn` e redireciona mesmo assim. A tela parece entrar, mas as chamadas seguintes falham.
- Textos sem acento: “Sessoes ativas”, “Exportacao pronta!”, “Confirmar exclusao” e “Configuracoes da conta” (no dashboard).

Comportamento diferente do épico, por decisão ou por evolução

- 1.2: com a confirmação de e-mail ligada, o Supabase não informa que o e-mail já existe, então “Este email já está cadastrado.” pode nunca aparecer.
- 1.3: depois do login, o usuário vai para `/onboarding`, que manda ao dashboard se o onboarding já tiver sido concluído.
- 1.7: a frase de confirmação é aceita com espaços nas pontas, porque o código aplica `trim()`.
- 2.6 e 2.7: quem cobra e tenta de novo é o Stripe, e o Porquinho só reage aos webhooks. Não existe job próprio às 2h UTC.
- 2.3: os lembretes são enviados às 9h no horário de America/Sao_Paulo, e não no fuso do usuário. Também são enviados quando o backend sobe.
- 2.5: dentro do período já pago, reativar só desfaz o cancelamento. A cobrança imediata só acontece pelo Checkout, quando a assinatura já expirou.
- 2.2: o preço no código é R$ 19,90, e não os cerca de R$ 42 citados no épico.
- 2.x: a mensagem do erro 402 é sempre a de fim do trial, mas o interceptor redireciona para `/assinatura` antes de ela aparecer.
- 3.6: o passo de importação do onboarding já usa o upload do Epic 5 e tem o botão “Continuar”. A versão sem upload, com aviso de “em breve”, foi substituída.
- 4.1: o link no dashboard se chama “Contas bancárias”.
- 4.6 e 4.7: as mensagens de 404 são mais específicas que as da spec. Também não dá para provocar “Período inválido” pela interface.
- 5.3: o banco detectado só aparece enquanto a importação está em processamento.
- 5.11: o padrão aprendido é a descrição inteira normalizada, sem casamento parcial.
- 6.2: a hierarquia tem 1 nível só, por decisão sua. O botão “Mover” fica desabilitado quando há menos de 2 grupos.

Não testável pela interface

- 1.5: expiração de sessões.
- 4.x: auditoria, `deleted_by` e o saldo inicial preservado. Confira no banco.

- **2.6: a renovação mensal real, que precisa de Test Clock do Stripe.**